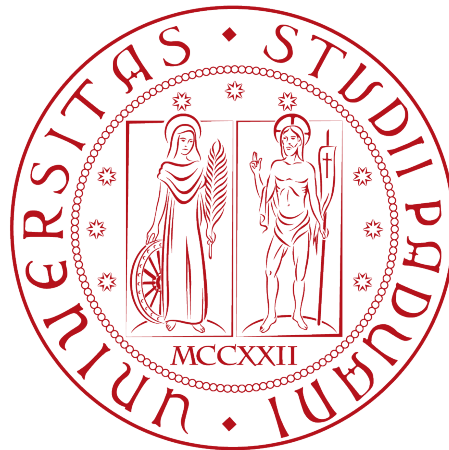


Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”

CORSO DI LAUREA IN INFORMATICA



Implementazione e Ottimizzazione di un Web
Application Firewall per la protezione di
applicazioni web

Tesi di laurea

Relatore

Prof. Davide Bresolin

Laureando

Andrea Perozzo

Matricola 2082849

ANNO ACCADEMICO 2024-2025

Indice

1	Introduzione	1
1.1	L'azienda	1
1.2	Obiettivo del progetto	2
1.3	Struttura del documento	2
2	Fondamenti teorici	3
2.1	Contesto delle applicazioni web	3
2.2	Sicurezza applicativa	4
2.2.1	Web Application Firewall: principi generali	4
2.2.2	Lo standard OWASP Top 10	5
2.2.3	OWASP Compliance	6
2.2.4	Obiettivo del progetto e approccio metodologico	7
2.3	Tecnologie e strumenti utilizzati	8
2.3.1	F5 BIG-IP come sistema di protezione applicativa	8
2.3.2	Strumenti a supporto del testing	9
2.4	Configurazione dell'ambiente virtuale	10
2.4.1	Panoramica delle macchine virtuali utilizzate	10
2.4.2	Schema di rete e topologia	11
2.4.3	Distribuzione e configurazione iniziale dell'ambiente	11
2.4.4	Primo test di integrazione tramite WAF	12
3	Configurazione e mitigazione delle vulnerabilità	14
3.1	Ambiente di lavoro e metodologia	14
3.1.1	Descrizione di OWASP NodeGoat	15
3.2	Mitigazione delle vulnerabilità secondo OWASP Top 10	15
3.2.1	A1 - Broken Access Control	16
3.2.2	A2 - Cryptographic Failures	17
3.2.3	A3 - Injection	19
3.2.4	A4 - Insecure Design	22
3.2.5	A5 - Security Misconfiguration	23
3.2.6	A6 - Vulnerable and Outdated Components	26
3.2.7	A7 - Identification and Authentication Failures	27
3.2.8	A8 - Software and Data Integrity Failures	29
3.2.9	A9 - Security Logging and Monitoring Failures	31
3.2.10	A10 - Server-Side Request Forgery (SSRF)	32
3.3	Risultato finale e osservazioni	34
4	Conclusioni	35

<i>INDICE</i>	iii
4.1 Riepilogo del lavoro svolto	35
4.2 Considerazioni tecniche	35
4.3 Considerazioni personali e professionali	36
4.4 Sviluppi futuri e possibili miglioramenti	36
Acronimi e abbreviazioni	38
Glossario	40

Elenco delle figure

1.1	Logo Azienda	1
2.1	Modello OSI a 7 livelli	4
2.2	Schema di rete e topologia delle macchine virtuali	11

Elenco delle tabelle

2.1	Categorie OWASP Top 10 (2021)	6
3.1	Controlli previsti per A1 - Broken Access Control	16
3.2	Controlli previsti per A2 - Cryptographic Failures	18
3.3	Controlli previsti per A3 - Injection	20
3.4	Controlli previsti per A4 - Insecure Design	22
3.5	Controlli previsti per A5 - Security Misconfiguration	24
3.6	Controlli previsti per A6 - Vulnerable and Outdated Components	26
3.7	Controlli previsti per A7 - Identification and Authentication Failures	28
3.8	Controlli previsti per A8 - Software and Data Integrity Failures	30
3.9	Controlli previsti per A9 - Security Logging and Monitoring Failures	31
3.10	Controlli previsti per A10 - Server-Side Request Forgery (SSRF)	33

Capitolo 1

Introduzione

Le applicazioni *web* rappresentano spesso l'anello più esposto verso l'esterno e, di conseguenza, uno dei principali vettori per attacchi informatici. Per mitigare tali minacce, i *Web Application Firewall (WAF)*^[g] costituiscono una soluzione efficace a livello applicativo, offrendo protezione contro vulnerabilità note.

Questo elaborato nasce a seguito di un periodo di *stage* della durata di 300 ore, svolto presso l'azienda. Durante questa esperienza, ho avuto l'opportunità di approfondire il funzionamento dei sistemi di protezione delle applicazioni *web*, concentrandomi in particolare sulla configurazione avanzata del *WAF* di *F5*^[g], con l'obiettivo di raggiungere il massimo livello di conformità secondo i criteri definiti da *Open Worldwide Application Security Project (OWASP)*^[g].

Il progetto si è articolato in due fasi principali. Nella prima fase, ho seguito un percorso formativo strutturato tramite l'esecuzione guidata di laboratori pratici, i quali mi hanno permesso di acquisire familiarità con l'ambiente di *test*, le tecnologie coinvolte e le funzionalità chiave del sistema. Nella seconda fase, ho operato in autonomia su un'applicazione *web* volutamente vulnerabile con l'obiettivo di identificare le vulnerabilità esistenti e di configurare in modo efficace il *firewall*^[g] per proteggerla.

1.1 L'azienda



Figura 1.1: Logo Azienda

Kirey è un *system integrator* e fornitore di soluzioni tecnologiche che opera a livello internazionale. Con sede a Padova e uffici dislocati in varie sedi italiane ed estere, l'azienda offre consulenza e servizi in ambiti quali *Digital Transformation*, *Cybersecurity*, *Big Data & Analytics*, *Cloud* e *Artificial Intelligence*.

Il rapporto con l'azienda e con il *tutor* aziendale è stato costante e costruttivo. Durante la prima parte dello *stage*, il *tutor* ha fornito supporto operativo e indicazioni metodologiche, facilitando l'apprendimento delle competenze tecniche. Nella fase successiva, il ruolo del *tutor* è stato più orientato al monitoraggio e alla validazione delle configurazioni da me proposte.

1.2 Obiettivo del progetto

L'obiettivo principale del progetto è stato quello di realizzare una configurazione di sicurezza in grado di proteggere un'applicazione *web* reale da attacchi informatici, mediante l'utilizzo del *WAF* di F5. Tale protezione è stata misurata e verificata in base al rispetto dei 10 livelli dello *standard OWASP*, ognuno dei quali copre una diversa categoria di vulnerabilità.

1.3 Struttura del documento

Il testo è organizzato in quattro capitoli principali:

Capitolo 2 presenta le tecnologie e i concetti teorici necessari alla comprensione del lavoro svolto, descrivendo l'ambiente configurato e gli *standard OWASP*.

Capitolo 3 espone il contributo originale, ovvero la configurazione del sistema di protezione della applicazione *web*, organizzato secondo i 10 livelli di *OWASP*.

Capitolo 4 riporta le conclusioni e le considerazioni finali sia dal punto di vista tecnico che personale.

Per facilitare la lettura, si segnala che:

- tutti i termini specialistici o ambigui sono definiti nel glossario presente in fondo al documento;
- la prima occorrenza dei termini presenti nel glossario è segnalata come segue: *FirstOccurrence*^[g];
- i termini stranieri e tecnici sono evidenziati in *corsivo*.

Capitolo 2

Fondamenti teorici

Questo capitolo presenta i concetti teorici e le tecnologie alla base del lavoro svolto durante il periodo di *stage*, fornendo una panoramica del contesto applicativo, delle vulnerabilità affrontate e degli strumenti utilizzati.

2.1 Contesto delle applicazioni web

Le applicazioni *web* moderne si basano sul modello *client-server*, che rappresenta lo schema fondamentale delle interazioni applicative: i *client* (utenti legittimi o potenziali attaccanti) inviano richieste al *server*, responsabile della gestione delle risorse e dell'esecuzione della logica applicativa. Proprio per questo il *server* costituisce il principale bersaglio di attacchi informatici che sfruttano la superficie di esposizione dell'applicazione, rendendo necessaria l'adozione di strumenti di protezione come i *WAF*, progettati per intercettare e bloccare traffico malevolo prima che raggiunga l'applicazione.

Negli ultimi anni, le *web app* si sono evolute verso architetture dinamiche e distribuite, basate su modelli a microservizi e soluzioni come *Single Page Application (SPA)* e *Progressive Web Application (PWA)*. Esse comunicano tipicamente tramite *Application Programming Interface (API)*, che permettono lo scambio di dati in formati standardizzati come *JavaScript Object Notation (JSON)* o *eXtensible Markup Language (XML)*, e sempre più spesso si avvalgono di protocolli come *WebSocket*, capaci di mantenere connessioni bidirezionali persistenti in tempo reale tra *client* e *server*. Questa complessità, se da un lato abilita funzionalità avanzate, dall'altro amplia le superfici di attacco: ogni componente, dall'interfaccia utente alla logica *server-side*, fino alle *API* e ai sistemi di *storage*, rappresenta un potenziale punto di ingresso per attori malevoli.

Il funzionamento delle applicazioni si fonda principalmente sui protocolli *HyperText Transfer Protocol (HTTP)*, *HTTP Secure (HTTPS)* e *WebSocket*, che costituiscono il canale di interazione tra *client* e *server*. *HTTP*, per sua natura *stateless* e privo di cifratura, risulta facilmente intercettabile e manipolabile; *HTTPS*, grazie all'integrazione con *Transport Layer Security (TLS)*, garantisce confidenzialità, autenticazione e integrità dei dati, ma rimane vulnerabile in caso di configurazioni deboli o certificati non validi. Infine, *WebSocket*, diffuso per applicazioni *real-time*, introduce nuove sfide di sicurezza, potendo essere sfruttato per attacchi come *message injection* o accessi non autorizzati a risorse interne.

Per queste ragioni, un *WAF* moderno deve essere in grado non solo di gestire correttamente tali protocolli, ma anche di ispezionare e filtrare il traffico applicativo che li utilizza, rilevando comportamenti anomali e prevenendo tentativi di sfruttamento delle vulnerabilità.

2.2 Sicurezza applicativa

Questa sezione presenta lo *standard OWASP Top 10*, punto di riferimento per la classificazione delle vulnerabilità più critiche e illustra il concetto di *OWASP compliance*, utilizzato nel contesto di questo progetto per valutare il livello di sicurezza raggiunto tramite la configurazione di un *WAF*.

2.2.1 Web Application Firewall: principi generali

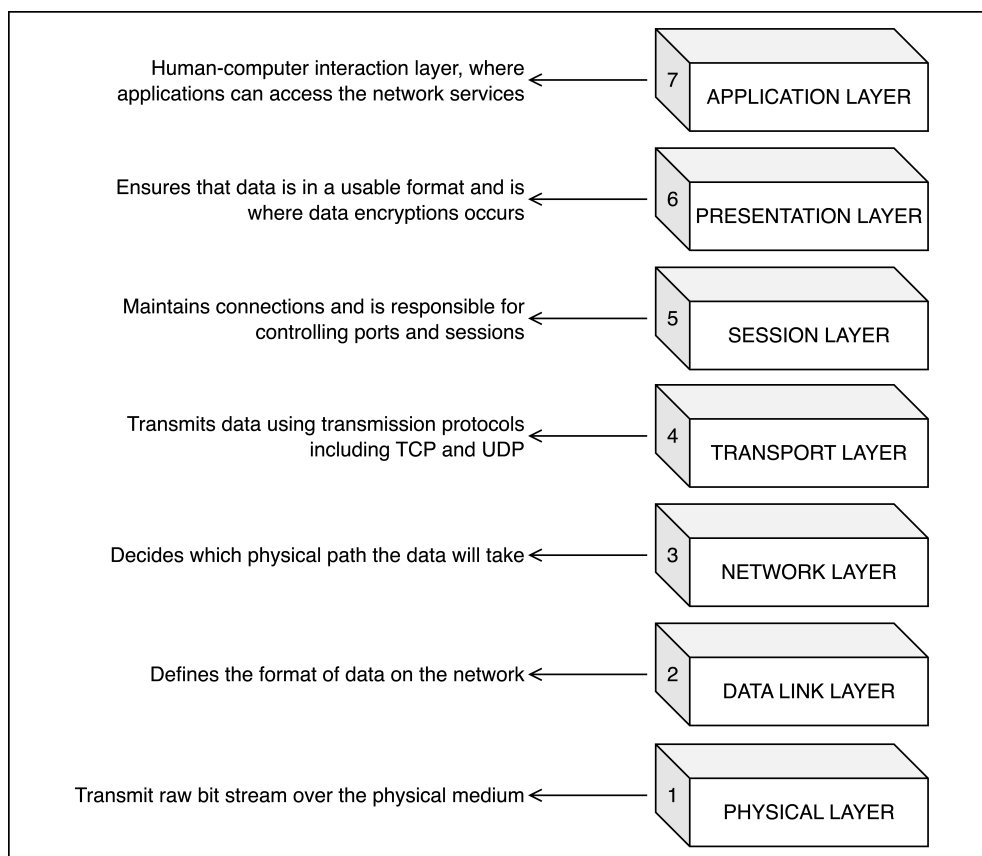


Figura 2.1: Modello OSI a 7 livelli

Un *WAF* è una soluzione di sicurezza progettata per monitorare, filtrare e bloccare il traffico *HTTP/HTTPS* diretto verso applicazioni *web*, con l'obiettivo di proteggere da attacchi a livello applicativo. A differenza dei *firewall* di rete tradizionali, che operano a livelli inferiori dello *stack Open Systems Interconnection (OSI)*^[g] mostrato nella [Figura 2.1](#) (tipicamente tra livello 3 e 4), un *WAF* lavora sul livello 7 (applicazione),

ispezionando il contenuto delle richieste e risposte per individuare comportamenti anomali o potenzialmente malevoli.

Il principio di funzionamento di un *WAF* si basa su una serie di regole e profili configurabili che permettono di:

- rilevare *pattern* noti di attacco tramite *signature*^[g] predefinite;
- analizzare e validare i parametri delle richieste (*Uniform Resource Locator (URL)*^[g], *header*^[g], *cookie*^[g], *payload*^[g]);
- applicare restrizioni su metodi *HTTP* ammessi, dimensione delle richieste, ecc.;
- profilare il traffico applicativo per rilevare comportamenti anomali (*Denial of Service (DoS)*^[g], *credential stuffing*^[g], *web scraping*^[g], ecc.);
- bloccare o registrare le richieste ritenute sospette, con azioni configurabili (es. *alert*, *block*, *redirect*, *log*^[g] *only*).

I *WAF* moderni operano tipicamente in una delle seguenti modalità:

- **Transparent Mode:** le richieste sospette vengono rilevate e tracciate nei *log*, ma non bloccate. Questa modalità è utile in fase di *test* e profilazione.
- **Blocking Mode:** il traffico considerato malevolo viene immediatamente respinto o reindirizzato, impedendo l'esecuzione dell'attacco.

L'utilizzo di un *WAF* non esclude la necessità di seguire buone pratiche di sviluppo sicuro o di adottare altre misure di sicurezza a livello di rete e sistema. Tuttavia, rappresenta uno strato difensivo in grado di intercettare molte delle vulnerabilità note prima che possano compromettere l'applicazione.

Nel contesto di questo progetto, il *WAF* utilizzato è stato quello integrato nella piattaforma *BIG-IP*^[g] di F5, che verrà descritto nel dettaglio nella [Sezione 2.3](#).

2.2.2 Lo standard OWASP Top 10

Lo *OWASP Top 10* è una classificazione internazionale, pubblicata periodicamente dalla *Open Worldwide Application Security Project*, che elenca le dieci categorie di vulnerabilità più critiche riscontrate nelle applicazioni *web*. Si tratta di uno *standard* riconosciuto e adottato sia in ambito accademico che industriale, utilizzato come strumento di sensibilizzazione e come guida tecnica per sviluppatori, amministratori e professionisti della sicurezza informatica.

L'obiettivo principale dello *OWASP Top 10* è duplice: aumentare la consapevolezza sui rischi legati alla sicurezza applicativa e fornire un punto di riferimento condiviso per identificare e affrontare le vulnerabilità più comuni.

L'ultima versione dello *standard*, pubblicata nel 2021, definisce le categorie di rischio mostrate nella [Tabella 2.1](#):

Codice	Categoria
A01	Broken Access Control: errori nella gestione dei privilegi di accesso che permettono operazioni non autorizzate.
A02	Cryptographic Failures: problemi nella protezione dei dati in transito o a riposo, come l'assenza di crittografia o l'uso di algoritmi obsoleti.
A03	Injection: esecuzione di comandi indesiderati nel sistema <i>target</i> , come <i>Structured Query Language (SQL)</i> ^[g] o comandi di <i>shell</i> ^[g] , dovuta a un' inadeguata validazione degli <i>input</i> .
A04	Insecure Design: assenza di principi di sicurezza nella fase di progettazione dell'applicazione.
A05	Security Misconfiguration: configurazioni errate o insicure di componenti <i>software</i> o infrastrutture.
A06	Vulnerable and Outdated Components: utilizzo di librerie o <i>framework</i> ^[g] con vulnerabilità note e non aggiornati.
A07	Identification and Authentication Failures: debolezze nei meccanismi di autenticazione e gestione delle sessioni.
A08	Software and Data Integrity Failures: mancata verifica dell'integrità del codice e dei dati, ad esempio in <i>pipeline</i> ^[g] <i>Continuous Integration (CI)</i> ^[g] / <i>Continuous Delivery (CD)</i> ^[g] o aggiornamenti <i>software</i> .
A09	Security Logging and Monitoring Failures: carenza di <i>log</i> , monitoraggio e capacità di risposta agli incidenti.
A10	Server-Side Request Forgery (SSRF): manipolazione della funzionalità <i>server-side</i> per eseguire richieste non autorizzate verso sistemi interni o esterni.

Tabella 2.1: Categorie OWASP Top 10 (2021)

Questo elenco non deve essere considerato esaustivo, ma come un punto di partenza per l'analisi e la mitigazione delle vulnerabilità applicative più rilevanti. La classificazione *OWASP*, infatti, viene periodicamente aggiornata per riflettere l'evoluzione delle minacce e delle tecnologie adottate nelle applicazioni *web*.

2.2.3 OWASP Compliance

Lo *standard OWASP Top 10* non definisce requisiti formali di conformità, ma si propone come strumento di sensibilizzazione e guida per la protezione delle applicazioni *web*. Alcuni produttori di soluzioni di sicurezza hanno tuttavia introdotto il concetto di *OWASP compliance* come metrica operativa, utile a valutare in modo oggettivo quanto un sistema sia protetto rispetto alle dieci categorie individuate da *OWASP*.

In questo contesto, la *compliance* non va intesa come certificazione ufficiale, bensì come un indicatore fornito da strumenti specifici. Essa consente di verificare se, per ciascuna categoria, siano attivi adeguati meccanismi di difesa, come controlli sugli *input*, protezioni per l'autenticazione, gestione dei *log* o limitazioni sul traffico anomalo. Tale interpretazione operativa ha reso l'*OWASP compliance* una metrica di riferimento utile in fase di *auditing*^[g] e di valutazione dei sistemi, fornendo un parametro sintetico per misurare il livello di sicurezza raggiunto e guidare le attività di miglioramento.

Il *WAF* di F5, tramite il modulo *Application Security Manager (ASM)*^[g], integra questa logica mettendo a disposizione una funzionalità dedicata denominata *OWASP*

Compliance. Essa permette di valutare in tempo reale il livello di copertura delle dieci categorie, verificando la presenza di specifici controlli attivi.

La *dashboard* del sistema mostra lo stato di ciascuna categoria con tre indicatori possibili:

- **Protected**, se i controlli richiesti risultano completi e attivi;
- **Partially Protected**, se solo una parte dei requisiti è soddisfatta;
- **Not Protected**, se la categoria non è coperta da alcun meccanismo.

Il punteggio complessivo, espresso in scala da 0 a 10, rappresenta il numero di categorie contrassegnate come *Protected* e costituisce un indicatore immediato del livello di sicurezza applicativa.

Nel [Capitolo 3](#) verranno analizzate nel dettaglio tutte le dieci categorie *OWASP*, con i *test* condotti, le configurazioni adottate e i risultati ottenuti fino al raggiungimento del punteggio massimo di *compliance*.

2.2.4 Obiettivo del progetto e approccio metodologico

L'obiettivo principale del progetto di *stage* era configurare un sistema di protezione capace di garantire la massima copertura rispetto alle dieci categorie di vulnerabilità definite nello *OWASP Top 10*. Per misurare il livello di sicurezza raggiunto è stato adottato l'indicatore di *OWASP compliance* fornito dal *WAF* di F5: un punteggio da 0 a 10 che rappresenta il numero di categorie effettivamente coperte dalla configurazione attiva. Il traguardo era ottenere il punteggio massimo di 10/10, dimostrando che l'applicazione vulnerabile *OWASP NodeGoat*^[g], inizialmente priva di protezioni, potesse essere difesa in modo completo.

Per raggiungere questo obiettivo è stato seguito un approccio iterativo articolato in tre fasi:

- **Verifica:** per ciascuna categoria *OWASP* è stata controllata la presenza di vulnerabilità in *NodeGoat*, eseguendo *test* manuali con strumenti come *Burp Suite*^[g] e simulando scenari di attacco realistici.
- **Mitigazione:** una volta confermata la vulnerabilità, la configurazione del *WAF* è stata adattata per bloccarne lo sfruttamento, utilizzando le funzionalità del modulo *ASM* (es. attivazione di firme specifiche, profili di sicurezza per *Brute Force*^[g] e *DoS*, controlli su parametri, *cookie* e *URL*). Le regole sono state applicate inizialmente in modalità *staging*^[g], per monitorarne l'impatto, e solo successivamente portate in *enforcement*^[g].
- **Validazione:** gli stessi attacchi sono stati ripetuti per verificare che le violazioni venissero bloccate dal *WAF*, che i *log* di sicurezza registrassero l'evento e che la *dashboard* di *OWASP Compliance* segnasse la categoria come coperta.

Questo metodo ha consentito di affinare progressivamente la configurazione, riducendo i falsi positivi e incrementando la copertura delle *policy*^[g], fino al raggiungimento della piena *OWASP compliance*.

2.3 Tecnologie e strumenti utilizzati

Questa sezione descrive la piattaforma *BIG-IP* di F5, utilizzata come componente centrale per la protezione dell'applicazione *web*. Vengono inoltre presentati i principali strumenti impiegati per simulare attacchi e valutare l'efficacia delle contromisure adottate.

2.3.1 F5 BIG-IP come sistema di protezione applicativa

BIG-IP è una piattaforma di sicurezza e bilanciamento del carico sviluppata da F5, utilizzata in ambito *enterprise* per garantire disponibilità, scalabilità e protezione delle applicazioni *web*. Tra i moduli che compongono la piattaforma, quello specificamente dedicato alla sicurezza applicativa è l'*ASM*, che implementa le funzionalità di *WAF*.

Il *ASM* consente di definire *policy* di sicurezza dettagliate che regolano l'accesso e l'uso delle applicazioni protette, monitorando il traffico *HTTP/HTTPS* in tempo reale e applicando una serie di controlli configurabili. I principali elementi che lo caratterizzano sono:

- **Signature e Rilevamento degli attacchi:** Il modulo include un *set* aggiornato di firme di attacco utilizzate per riconoscere schemi noti di attacchi come *SQL injection (SQLi)*^[g], *Cross-Site Scripting (XSS)*^[g], *command injection*^[g], e altri. Ogni firma può essere attivata, disattivata o personalizzata in base al contesto dell'applicazione da proteggere.
- **Staging ed enforcement:** Le *policy* possono inizialmente operare in *Transparent Mode (staging)*, in cui le violazioni vengono tracciate nei *log* ma non ancora bloccate. Questo consente di affinare le configurazioni prima di attivare la protezione effettiva. Successivamente, le *policy* possono essere applicate in *Blocking Mode (enforcement)*, in cui le richieste sospette vengono attivamente respinte o gestite con regole definite.
- **Validazione del traffico:** È possibile specificare regole per la validazione degli elementi delle richieste, come *URL*, parametri, *header*, *cookie* e metodi *HTTP*. Il sistema consente di configurare restrizioni su:
 - parametri ammessi (nome, tipo, lunghezza, caratteri validi);
 - metodi *HTTP* consentiti (*GET*^[g], *POST*^[g]);
 - *path* e *file* accessibili;
 - intestazioni e attributi dei *cookie* (es. *HttpOnly*^[g]).
- **Logging e notifiche:** Ogni evento rilevante viene tracciato nei *log* applicativi, consultabili dall'interfaccia *web* tramite la sezione *Event Logs*. È possibile filtrare i *log* per tipo di violazione, indirizzo *Internet Protocol (IP)*^[g], *URL* richiesto e firma attivata, rendendo più agevole l'analisi delle minacce e la messa a punto delle *policy*.
- **Dashboard e compliance:** Il modulo include una sezione dedicata alla valutazione della *OWASP Compliance*, che permette di monitorare graficamente il livello di protezione raggiunto rispetto alle dieci categorie dello *standard OWASP Top 10*. Questo strumento è utile per validare l'efficacia delle configurazioni e identificare rapidamente eventuali lacune.

L'interfaccia *web-based* del sistema, consente di gestire in modo centralizzato tutti gli aspetti operativi del *WAF*, dalla definizione delle regole al monitoraggio delle violazioni, rendendo la piattaforma F5 uno strumento potente e flessibile per la protezione delle applicazioni *web*.

2.3.2 Strumenti a supporto del testing

Durante le fasi di apprendimento e configurazione del *WAF*, sono stati utilizzati diversi strumenti per testare il comportamento delle applicazioni *web*, simulare attacchi e verificare l'efficacia delle *policy* di sicurezza adottate. In questa sezione vengono presentati i principali strumenti impiegati:

- **Burp Suite:** Uno dei *tool* più diffusi per il *penetration testing* di applicazioni *web*. Sviluppato da [PortSwigger](#)^[g], offre una *suite* integrata di strumenti per l'analisi e la manipolazione del traffico *HTTP/HTTPS*. Le sue funzionalità principali includono:
 - un *proxy*^[g] intercettatore, che consente di catturare, modificare e inoltrare le richieste tra *browser* e *server*;
 - uno *scanner* automatico, utile per individuare vulnerabilità comuni come *XSS*, *SQLi*, *Cross-Site Request Forgery (CSRF)*^[g];

Nel progetto, *Burp Suite* è stato utilizzato per generare richieste malformate verso le applicazioni vulnerabili e osservare come venivano trattate dal *WAF*. Il traffico intercettato è stato fondamentale per comprendere la logica delle richieste *HTTP* e per testare la risposta del sistema in presenza di *payload* malevoli.

- **OWASP Juice Shop:** *Juice Shop*^[g] è una delle applicazioni volutamente vulnerabili più complete, progettata per simulare scenari di attacco reali. Include vulnerabilità riconducibili a tutte le voci dell'*OWASP Top 10*, come:
 - *injection* di comandi *SQL* e *JavaScript*^[g];
 - violazioni dell'*access control*^[g];
 - esposizione di dati sensibili;
 - meccanismi di autenticazione deboli;
 - configurazioni errate e attacchi *client-side*.

Nel progetto, *Juice Shop* è stata utilizzata come *web application target* durante la prima fase dei laboratori formativi. L'obiettivo era familiarizzare con il comportamento di un'applicazione vulnerabile e osservare come il *WAF* reagisse ai diversi tipi di attacco.

- **OWASP NodeGoat:** *NodeGoat* è un'altra applicazione *open source* progettata da *OWASP* per l'insegnamento della sicurezza *web*. Sviluppata con [Node.js](#)^[g] e [MongoDB](#)^[g], offre un ambiente più leggero e modulare rispetto a *Juice Shop*, con vulnerabilità mirate a categorie specifiche come:
 - *NoSQL injection*^[g];
 - *broken access control*;
 - gestione non sicura dei *token*^[g] e dei *cookie*;

- attacchi basati su sessioni e permessi.

Durante la seconda fase dello *stage*, *NodeGoat* è stata utilizzata come riferimento per la configurazione autonoma del *WAF*. L'intero progetto di mitigazione delle vulnerabilità e raggiungimento della *OWASP compliance* 10/10 è stato condotto su questa piattaforma.

2.4 Configurazione dell'ambiente virtuale

In questa sezione viene descritta la composizione dell'ambiente virtuale, con particolare attenzione alle macchine coinvolte, alla topologia di rete utilizzata e alla configurazione iniziale del sistema. Viene inoltre documentato il primo *test* di integrazione, che ha verificato il corretto instradamento del traffico attraverso il *WAF* e la registrazione degli eventi di sicurezza.

2.4.1 Panoramica delle macchine virtuali utilizzate

Per lo svolgimento delle attività pratiche previste durante lo *stage*, è stato predisposto un ambiente virtuale composto da tre macchine virtuali distinte, ciascuna con un ruolo specifico. Le macchine sono state eseguite tramite [VMware Workstation](#)^[g], che ha consentito di simulare un'infrastruttura di rete realistica in grado di supportare attività di *test* e configurazione in completa autonomia.

- **Ubuntu Server:** macchina virtuale basata su [Ubuntu](#)^[g] *Server 22.04 LTS*, utilizzata per ospitare le *web application* vulnerabili (*OWASP Juice Shop* e *OWASP NodeGoat*). L'ambiente è stato configurato per eseguire i [container](#)^[g] tramite [Docker](#)^[g], consentendo il rapido avvio e ripristino delle applicazioni *target*.
- **Ubuntu Client:** macchina virtuale con sistema operativo *Ubuntu Desktop*, utilizzata per simulare il comportamento di un utente esterno e per eseguire gli strumenti di analisi e attacco, tra cui *Burp Suite*. Inoltre, questa macchina ha svolto il ruolo di interfaccia operativa per accedere alla console *web* del *WAF* tramite *browser*.
- **F5 BIG-IP:** macchina virtuale fornita dall'azienda, contenente la piattaforma *BIG-IP* con il modulo *ASM* attivo. Questa macchina ha rappresentato il nodo centrale del sistema di protezione, svolgendo il ruolo di *WAF* tra il *client* e il *server*.

L'adozione di un'infrastruttura virtuale ha consentito di:

- isolare le componenti del sistema su sottoreti logiche dedicate;
- eseguire *test* in un ambiente controllato e riproducibile;
- verificare il comportamento del traffico in transito attraverso il *WAF*;
- ripristinare facilmente lo stato iniziale delle macchine in caso di errori o modifiche non desiderate.

Questa configurazione ha costituito la base operativa per tutte le attività successive di configurazione, simulazione di attacchi e analisi dei *log*.

2.4.2 Schema di rete e topologia

Le tre macchine virtuali utilizzate sono state collegate a una rete logica composta da tre sottoreti distinte, ciascuna con un ruolo specifico all'interno dell'infrastruttura (vedi Figura 2.2). La configurazione è stata effettuata manualmente tramite *VMware Workstation*, assegnando a ciascuna macchina virtuale una o più interfacce di rete in base alla funzione ricoperta. Questo schema ha permesso di simulare una topologia realistica, con separazione tra traffico interno, esterno e di gestione.

Le sottoreti configurate sono le seguenti:

- **Rete management (10.1.1.0):** utilizzata esclusivamente per accedere all'interfaccia di gestione del *WAF* tramite *browser* dalla macchina *Ubuntu Client*.
- **Rete external (10.1.10.0):** rappresenta la rete pubblica, ovvero l'interfaccia attraverso cui il *client* accede all'applicazione protetta. Il traffico generato su questa rete viene filtrato dal *WAF*.
- **Rete internal (10.1.20.0):** rete privata usata per connettere il *WAF* al *server* che ospita l'applicazione vulnerabile. Tutte le richieste indirizzate all'applicazione passano attraverso questa rete.

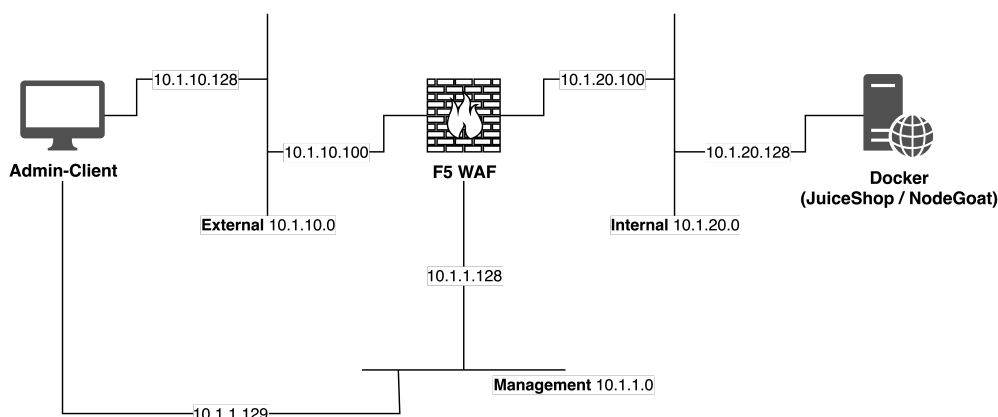


Figura 2.2: Schema di rete e topologia delle macchine virtuali

Questa suddivisione permette di emulare fedelmente una configurazione a tre livelli, in cui il *WAF* funge da *proxy* inverso che intercetta e filtra tutto il traffico proveniente dall'esterno prima che raggiunga l'applicazione. Inoltre, consente di isolare la gestione amministrativa del sistema da eventuali attacchi generati dal *client*.

2.4.3 Distribuzione e configurazione iniziale dell'ambiente

La configurazione iniziale dell'ambiente virtuale è stata un passaggio fondamentale per poter eseguire correttamente i *test* di sicurezza e sperimentare le funzionalità offerte dal *WAF*. Il processo è stato realizzato manualmente, a partire dal *download* delle immagini delle macchine virtuali, fino alla configurazione degli indirizzi *IP* e al *deploy* della prima *web application* vulnerabile.

- **Installazione delle macchine virtuali:** L'ambiente è stato predisposto nel seguente modo:

- La macchina *F5 BIG-IP* è stata fornita in formato *virtual appliance* dall'azienda ospitante ed è stata importata direttamente in *VMware Workstation*.
- Le macchine *Ubuntu Server* e *Ubuntu Client* sono state create da zero a partire da immagini ufficiali scaricate dal sito di *Ubuntu*.
- **Configurazione delle interfacce di rete:** Ogni macchina virtuale è stata collegata alle reti logiche descritte nella sezione precedente, e gli indirizzi *IP* statici sono stati configurati come segue:
 - **Ubuntu Server** è stato collegato alla seguente rete:
 - * *internal* (10.1.20.0), con indirizzo 10.1.20.128.
 - **Ubuntu Client** è stato collegato a due reti:
 - * *management* (10.1.1.0), con indirizzo 10.1.1.129;
 - * *external* (10.1.10.0), con indirizzo 10.1.10.128.
 - **F5 BIG-IP** è stato configurato con tre interfacce di rete:
 - * *management* (10.1.1.0), con indirizzo 10.1.1.128;
 - * *external* (10.1.10.0), con indirizzo 10.1.10.100;
 - * *internal* (10.1.20.0), con indirizzo 10.1.20.100.
- **Deploy della web application vulnerabile:** Sulla macchina *Ubuntu Server* è stata avviata la prima applicazione vulnerabile utilizzando *Docker*. In particolare:
 - è stato eseguito il comando `docker pull bkimminich/juice-shop` per scaricare l'immagine di *OWASP Juice Shop*;
 - successivamente, l'*app* è stata avviata con il comando `docker run -d -p 3000:3000 bkimminich/juice-shop`;
 - il servizio è stato così esposto all'indirizzo `http://10.1.20.128:3000/`, accessibile solo tramite la rete interna.
- **Accesso all'interfaccia di gestione F5:** Dopo aver avviato correttamente la macchina virtuale F5, è stato possibile accedere alla *console web* di configurazione dal *browser* della macchina *Ubuntu Client*, all'indirizzo `https://10.1.1.128/`. L'interfaccia fornita da F5 ha consentito di configurare i nodi, le *pool*^[g] e le *policy* di sicurezza, oltre a monitorare il traffico applicativo in tempo reale.

Questa fase ha permesso di ottenere un'infrastruttura funzionante, nella quale il traffico *client-server* passava obbligatoriamente attraverso il *WAF*, creando così le condizioni ottimali per procedere alle attività di *tuning*^[g], *test* delle vulnerabilità e analisi dei *log*.

2.4.4 Primo test di integrazione tramite WAF

Una volta completata la configurazione dell'ambiente virtuale e avviata correttamente la *web application* vulnerabile *OWASP Juice Shop* sulla macchina *Ubuntu Server*, è stato possibile procedere con il primo *test* di integrazione attraverso il *WAF*. Questa fase aveva lo scopo di verificare che il traffico generato dal *client* passasse correttamente attraverso il *firewall* e venisse tracciato nei *log* applicativi.

- **Creazione del nodo e della pool:** Accedendo all'interfaccia di gestione di F5 dalla macchina *Ubuntu Client* tramite *browser* `https://10.1.1.128/`, è stata configurata la sezione *Local Traffic*, in particolare:

- è stato creato un nuovo nodo con indirizzo *IP* 10.1.20.100 e porta 3000, corrispondente al servizio *Juice Shop* in esecuzione su *Ubuntu Server*;
- successivamente, è stata definita una *pool* contenente il nodo appena creato, che rappresenta il gruppo di *server* a cui inoltrare le richieste.
- **Creazione del virtual server:** Per rendere raggiungibile l'applicazione dal *client*, è stato creato un *Virtual Server*^[6] associato alla rete *external*, con indirizzo *IP* 10.1.10.101 e porta 80, che funge da punto di ingresso pubblico per l'applicazione. Tutto il traffico diretto verso questo indirizzo viene processato dal *WAF* prima di essere inoltrato alla *pool* interna.
- **Verifica del flusso di traffico:** Dalla macchina *Ubuntu Client*, l'utente ha potuto accedere all'applicazione digitando l'indirizzo `http://10.1.10.101/` nel *browser*. Il corretto caricamento della pagina *Juice Shop* ha confermato che il flusso di rete seguiva il percorso atteso:

Client → *WAF* → *Web Server*

- **Consultazione dei log:** Infine, sono stati esaminati i *log* generati dal *WAF* nella sezione *Event Logs*. Qui era possibile visualizzare i dettagli delle richieste *HTTP* processate, tra cui:
 - indirizzo *IP* del *client*;
 - *URL* richiesto;
 - parametri inviati;
 - eventuali violazioni rilevate dalle firme attive;
 - azione eseguita dal sistema (es. *alarm*, *block*, *learn*).

Questo primo *test* ha confermato che il *WAF* era correttamente posizionato tra *client* e *server*, in grado di intercettare e ispezionare il traffico applicativo. Ha inoltre rappresentato il punto di partenza per tutte le successive attività di simulazione di attacchi e verifica della *OWASP compliance*.

Capitolo 3

Configurazione e mitigazione delle vulnerabilità

In questo capitolo verranno presentati nel dettaglio i passi seguiti per ogni livello *OWASP*, illustrando gli attacchi condotti su *NodeGoat*, le vulnerabilità riscontrate, le regole di sicurezza configurate nel *firewall* e i *test* di verifica che hanno confermato il raggiungimento dell'obiettivo di protezione.

3.1 Ambiente di lavoro e metodologia

Il percorso di *stage* è iniziato con una fase di formazione strutturata, della durata di due settimane, basata sull'esecuzione di laboratori pratici forniti dall'azienda. Queste attività sono state svolte utilizzando come applicazione *target OWASP Juice Shop*.

I laboratori hanno avuto lo scopo di fornire una panoramica completa delle principali funzionalità del *WAF* di F5, consentendo di acquisire familiarità con molte delle interfacce disponibili nel sistema di gestione. In particolare, ho potuto sperimentare in modo diretto:

- la creazione, modifica e gestione delle *policy* di sicurezza, incluse le impostazioni generali e le configurazioni avanzate;
- l'attivazione e la personalizzazione delle *attack signatures*, comprese le logiche di apprendimento (*Traffic Learning*) e le modalità *staging* ed *enforcement*;
- la definizione di regole per *URL* consentiti o bloccati, per parametri e caratteri ammessi, e per la gestione di *file types*, *header* e *cookie*;
- l'utilizzo di funzionalità di protezione avanzata come *Data Guard*^[g], *Brute Force Prevention*, *Bot*^[g] *Defense* e *IP Intelligence*^[g];
- la configurazione delle pagine di blocco e risposta, con possibilità di personalizzare i messaggi restituiti al *client* in caso di violazione;
- l'analisi e il monitoraggio tramite *Event Logs*, con visibilità sugli eventi relativi a richieste applicative, attacchi *DoS*, traffico *bot* e *policy* di sicurezza;
- la configurazione di componenti di rete (Nodi, *Pools*, *Virtual Servers*), necessarie per instradare correttamente il traffico attraverso il *WAF*.

Questa fase ha permesso di comprendere in modo pratico come identificare vulnerabilità tipiche, simulare attacchi tramite strumenti come *Burp Suite* e, soprattutto, configurare il *firewall* per mitigare i rischi riscontrati. Ogni laboratorio seguiva una sequenza definita: individuazione della vulnerabilità, applicazione della contromisura tramite *policy* del *WAF*, e infine verifica del corretto blocco dell'attacco.

Terminate le attività di formazione, il lavoro è proseguito in autonomia su *OWASP NodeGoat*. In questa seconda fase, l'obiettivo non era più quello di seguire esercitazioni guidate, ma di applicare in modo indipendente le conoscenze acquisite, configurando e combinando le varie funzionalità del *WAF* per raggiungere la massima *OWASP compliance*. *NodeGoat* ha quindi rappresentato il banco di prova per validare in autonomia quanto appreso durante i laboratori.

3.1.1 Descrizione di OWASP NodeGoat

OWASP NodeGoat è un'applicazione *web* vulnerabile sviluppata e mantenuta dalla *Open Worldwide Application Security Project (OWASP)*. Realizzata in *Node.js* e basata su database *MongoDB*, *NodeGoat* nasce con lo scopo didattico di fornire un ambiente realistico per lo studio delle vulnerabilità applicative più comuni.

L'applicazione riproduce uno scenario gestionale semplice, con funzionalità di autenticazione, registrazione utenti e gestione di oggetti, ed è progettata per includere numerose debolezze di sicurezza intenzionali. Tra le principali categorie di vulnerabilità presenti si trovano:

- *Injection (SQL/NoSQL, Command, LDAP, XPath)*;
- *Cross-Site Scripting (XSS)*;
- *Cross-Site Request Forgery (CSRF)*;
- *Broken Access Control*;
- *Session hijacking* e debolezze nei meccanismi di autenticazione;
- Esposizione di dati sensibili e configurazioni insicure.

La caratteristica distintiva di *NodeGoat* è quella di essere allineato con le dieci categorie dello *OWASP Top 10*, rendendolo un banco di prova ideale per esercitazioni pratiche e per la validazione di contromisure di sicurezza.

Nel contesto di questo progetto di *stage*, *NodeGoat* è stato scelto come applicazione *target* per la fase autonoma, poiché ha permesso di applicare in maniera estesa le funzionalità del *WAF* e di verificare il raggiungimento della piena *OWASP compliance* (10/10) in scenari concreti di vulnerabilità note.

3.2 Mitigazione delle vulnerabilità secondo OWASP Top 10

Questa sezione illustra in dettaglio le attività svolte per analizzare, mitigare e validare le vulnerabilità presenti nell'applicazione *NodeGoat*, con l'obiettivo di raggiungere una *OWASP compliance* pari a 10/10.

3.2.1 A1 - Broken Access Control

La categoria *Broken Access Control* comprende tutte quelle vulnerabilità legate a una gestione inadeguata dei meccanismi di autenticazione e autorizzazione. Un'applicazione vulnerabile può consentire ad un utente non autorizzato di accedere a risorse riservate o di eseguire operazioni che dovrebbero essere vietate. Nel contesto della *OWASP compliance* di F5, questa categoria viene considerata coperta se la *policy* include controlli elencati nella [Tabella 3.1](#):

Controllo	Descrizione
Authentication / Authorization Attacks	Protezione da tentativi di aggirare i meccanismi di autenticazione e autorizzazione.
Path Traversal Detection	Individuazione di richieste che cercano di accedere a <i>directory</i> o <i>file</i> non autorizzati tramite sequenze come <code>[. ./]</code> .
Predictable Resource Location	Prevenzione di accessi a risorse o <i>file</i> con nomi prevedibili non destinati all'uso pubblico.
Login Enforcement	Obbligo di passare per la procedura di <i>login</i> definita, impedendo accessi diretti a risorse protette.
Disallowed File Types	Blocco di estensioni di <i>file</i> potenzialmente pericolose (es. <code>.exe</code> , <code>.bat</code> , <code>.php</code>).
Disallowed URLs	Blocco di percorsi e <i>URL</i> non autorizzati, definiti esplicitamente come non consentiti nella <i>policy</i> .

Tabella 3.1: Controlli previsti per A1 - Broken Access Control

- Verifica:** Durante la fase di verifica sono stati condotti *test* per valutare il comportamento di *NodeGoat* rispetto ai meccanismi di accesso e autorizzazione. In primo luogo, sono stati tentati accessi diretti a risorse protette senza una sessione valida, così da verificare l'assenza di un adeguato *login enforcement*. Sono stati poi manipolati gli *URL* e i parametri delle richieste per simulare attacchi di [path traversal](#)^[g], con l'obiettivo di raggiungere *directory* e *file* al di fuori dei percorsi consentiti. Parallelamente, si è verificata la possibilità di accedere a risorse tramite percorsi prevedibili o mediante *URL* esplicitamente vietati dalla *policy*. Un'ulteriore serie di prove ha riguardato il caricamento e l'accesso a *file* con estensioni non autorizzate, nonché tentativi di aggirare i controlli di autenticazione e autorizzazione attraverso manipolazioni delle sessioni e delle credenziali.
- Mitigazione:** Per ridurre le vulnerabilità rilevate, il *WAF* è stato configurato con una serie di contromisure mirate. In particolare, è stato abilitato il modulo di *Login Enforcement*, così da obbligare l'utente a passare attraverso la pagina di autenticazione prevista e impedire l'accesso diretto a risorse protette senza sessione valida. Sono stati inoltre definiti gli *Allowed URLs*, bloccando di conseguenza tutti i percorsi non autorizzati, e configurate regole per vietare il caricamento o l'accesso a *file* con estensioni non consentite. Parallelamente, sono state attivate le *signature* di rilevamento del *path traversal*, capaci di intercettare richieste che tentano di accedere a *directory* esterne ai percorsi legittimi.

Tutte queste configurazioni sono state inizialmente applicate in modalità *staging*, così da analizzare i *log* e ridurre al minimo i falsi positivi, e solo successivamente portate in *blocking mode*, garantendo il blocco effettivo delle richieste malevole.

- **Validazione:** Dopo l'attivazione delle contromisure, i *test* di attacco iniziali sono stati ripetuti per verificarne l'efficacia. I tentativi di accesso diretto a risorse protette senza autenticazione venivano intercettati e bloccati dalla funzionalità di *Login Enforcement*, impedendo così l'aggiramento dei meccanismi di autenticazione e autorizzazione. Analogamente, gli attacchi di *path traversal* producevano violazioni registrate nei *log* ed erano sistematicamente respinti dal WAF. Le richieste che cercavano di raggiungere percorsi non autorizzati o prevedibili venivano negate grazie alle regole sugli *Allowed/Disallowed URLs*, mentre i tentativi di caricare o accedere a *file* con estensioni vietate venivano bloccati dalla configurazione sui *Disallowed File Types*.

Il monitoraggio tramite la sezione *Event Logs* ha confermato l'attivazione corretta delle *policy*, mostrando per ogni attacco la rilevazione e l'azione di blocco intrapresa. In parallelo, la *dashboard* di *OWASP Compliance* ha indicato come pienamente coperta la categoria A1, segnando il contributo al punteggio complessivo e validando in maniera oggettiva l'efficacia delle misure adottate.

3.2.2 A2 - Cryptographic Failures

La categoria *Cryptographic Failures* riguarda tutte le problematiche legate a una gestione inadeguata dei dati sensibili, sia in transito che a riposo. Rientrano in questa classe l'assenza di crittografia, l'utilizzo di algoritmi deboli o obsoleti e la mancata protezione di informazioni riservate nei *log* e nelle comunicazioni. Nel contesto della *OWASP compliance* di F5, una *policy* viene considerata conforme se include i meccanismi mostrati nella [Tabella 3.2](#):

Controllo	Descrizione
Predictable Resource Location	Prevenzione dell'accesso a risorse sensibili con nomi o percorsi facilmente indovinabili.
Information Leakage	Protezione da divulgazione involontaria di informazioni riservate o dettagli tecnici nelle risposte.
Data Guard	Mascheramento automatico di dati sensibili (es. numeri di carte di credito) presenti nelle richieste o nelle risposte.
Masked Credit Card Numbers in Request Log	Offuscamento dei numeri di carte di credito nei <i>log</i> generati dal sistema, per evitare esposizioni accidentali.
Masked Parameters	Mascheramento del contenuto di specifici parametri nelle richieste, impedendone la registrazione in chiaro.
Masked Headers	Mascheramento dei valori di <i>header HTTP</i> sensibili (es. <i>header</i> di autenticazione) nei <i>log</i> applicativi.
Masked Cookies	Mascheramento dei <i>cookie</i> di sessione o di autenticazione nei <i>log</i> , per impedire furti o usi impropri.
Client-Side Encryption	Obbligo di utilizzo di protocolli sicuri (es. <i>HTTPS/TLS</i>) per garantire la cifratura dei dati in transito.
Server-Side Encryption	Cifratura dei dati persistenti o sensibili lato <i>server</i> , per proteggerli da accessi non autorizzati o compromissioni.

Tabella 3.2: Controlli previsti per A2 - Cryptographic Failures

- **Verifica:** Durante la fase di verifica, i *test* condotti su *NodeGoat* hanno evidenziato diverse criticità legate alla gestione dei dati sensibili. Alcuni parametri venivano trasmessi in chiaro, rivelando informazioni riservate che avrebbero potuto essere facilmente intercettate. Nei *log* generati dall'applicazione comparivano interi numeri di carta di credito, *cookie* di sessione e altri valori sensibili non mascherati, a dimostrazione dell'assenza di meccanismi come il *Data Guard* o il mascheramento di parametri, *header* e *cookie*.

Parallelamente, l'applicazione mostrava casi di *information leakage*, restituendo nelle risposte dettagli tecnici utili a un potenziale attaccante. Inoltre, non vi era alcun vincolo all'utilizzo del protocollo sicuro *HTTPS*, con il risultato che il traffico poteva essere trasmesso in chiaro ed essere soggetto a intercettazioni o manipolazioni. Infine, non erano presenti garanzie di cifratura dei dati sensibili lato *server*, rendendo vulnerabile anche la loro memorizzazione persistente.

- **Mitigazione:** Per ridurre i rischi individuati, il *WAF* è stato configurato con una serie di contromisure mirate alla protezione dei dati sensibili. In primo luogo è stata abilitata la funzionalità *Data Guard*, che maschera automaticamente stringhe numeriche riconducibili a numeri di carte di credito presenti nelle richieste e nelle risposte. Sono stati inoltre definiti parametri, *header* e *cookie* da mascherare, così da impedire che valori sensibili venissero registrati in chiaro nei *log* di sistema. Parallelamente è stata attivata l'opzione di protezione da *information leakage*, che blocca o filtra le risposte contenenti dettagli tecnici sull'applicazione o sull'infrastruttura.

Un'ulteriore misura ha riguardato l'imposizione dell'utilizzo del protocollo *HTTPS* per tutte le comunicazioni tra *client* e *server*, così da garantire la cifratura dei dati in transito ed evitare intercettazioni sul traffico. Infine, lato *server*, sono state abilitate le opzioni di cifratura per la protezione delle informazioni persistenti, assicurando che anche i dati archiviati non fossero accessibili in chiaro.

Come per le altre categorie, queste configurazioni sono state applicate inizialmente in modalità *staging*, per verificarne l'efficacia e individuare eventuali falsi positivi, e successivamente portate in *blocking mode*, così da garantire la piena protezione in produzione.

- **Validazione:** Al termine della fase di configurazione, i *test* sono stati ripetuti per verificare l'efficacia delle misure adottate. Nei *log* generati dal *WAF*, i numeri di carte di credito e i valori sensibili presenti nei parametri, negli *header* e nei *cookie* apparivano correttamente mascherati, impedendone la visualizzazione in chiaro. Le risposte restituite dall'applicazione non contenevano più messaggi di *debug* o informazioni architetturali, segno che la protezione contro l'*information leakage* era attiva e funzionante. Inoltre, i tentativi di accesso tramite protocollo non cifrato *HTTP* venivano respinti o automaticamente reindirizzati su *HTTPS*, garantendo così la cifratura *end-to-end*¹ delle comunicazioni.

Anche le opzioni di cifratura lato *server* sono risultate efficaci: i dati persistenti non erano più accessibili in chiaro, riducendo il rischio di compromissione in caso di accesso non autorizzato al *database*. La sezione *Event Logs* ha registrato ogni violazione intercettata, confermando la corretta applicazione delle regole, mentre la *dashboard* di *OWASP Compliance* ha segnato la categoria A2 come pienamente coperta, contribuendo in maniera significativa al raggiungimento del punteggio complessivo.

3.2.3 A3 - Injection

La categoria *Injection* comprende una vasta gamma di vulnerabilità in cui l'attaccante riesce a inserire o manipolare istruzioni malevole all'interno di un interprete, inducendo il sistema a eseguire comandi non previsti. Nell'ambito della *OWASP compliance* di F5, questa categoria è considerata coperta se la *policy* prevede il rilevamento e il blocco degli attacchi presenti nella [Tabella 3.3](#):

Controllo	Descrizione
Buffer Overflow	Protezione da <i>input</i> eccessivi che mirano a sovrascrivere la memoria del <i>server</i> , potenzialmente consentendo l'esecuzione di codice arbitrario.
Command Execution	Rilevamento e blocco di <i>input</i> che tentano di eseguire comandi di sistema sul <i>server</i> .
Cross-Site Scripting (XSS)	Prevenzione di iniezione di <i>script</i> malevoli nelle pagine <i>web</i> , sfruttati per rubare <i>cookie</i> o manipolare contenuti.
Denial of Service	Limitazione di <i>input</i> o richieste malevole progettate per sovraccaricare il <i>server</i> e renderlo indisponibile.
LDAP Injection	Protezione contro l'iniezione di comandi malevoli in query ^[g] <i>Lightweight Directory Access Protocol (LDAP)</i> ^[g] , usata per manipolare <i>directory</i> di autenticazione.
Path Traversal	Individuazione di richieste che cercano di accedere a <i>file</i> o <i>directory</i> al di fuori dei percorsi autorizzati tramite sequenze come <code>[. ./]</code> .
Remote File Include	Prevenzione di richieste che tentano di includere <i>file</i> remoti all'interno dell'applicazione per eseguire codice non autorizzato.
SQL Injection	Protezione contro l'iniezione di comandi <i>SQL</i> malevoli in <i>query</i> al <i>database</i> , che possono alterare o compromettere i dati.
Server-Side Code Injection	Rilevamento e blocco di tentativi di inserire ed eseguire codice lato <i>server</i> .
Vulnerability Scan	Identificazione di <i>pattern</i> tipici utilizzati dagli <i>scanner</i> automatici di vulnerabilità per individuare punti deboli.
XPath Injection	Prevenzione di iniezioni malevole in <i>query XML Path Language (XPath)</i> ^[g] , che consentono l'accesso non autorizzato a dati <i>XML</i> .
Evasion Techniques	Rilevamento di tecniche utilizzate per aggirare i controlli di sicurezza, come encoding ^[g] o obfuscation ^[g] .
Disallowed Meta Characters in Parameters	Blocco di caratteri speciali vietati nei parametri delle richieste (es. <code><</code> , <code>></code> , <code>;</code>) usati per iniezioni.
HttpOnly Cookie Attribute Enforcement	Applicazione dell'attributo <i>HttpOnly</i> ai <i>cookie</i> , impedendo l'accesso via <i>JavaScript</i> e riducendo l'impatto degli attacchi <i>XSS</i> .

Tabella 3.3: Controlli previsti per A3 - Injection

- **Verifica:** Nella fase di verifica, su *NodeGoat* sono stati condotti numerosi *test* di attacco per valutare la presenza di vulnerabilità riconducibili alla categoria delle *injection*. In diversi campi di *input* è stato possibile iniettare comandi *SQL*, ottenendo errori di *database* e accessi non autorizzati ai dati, a conferma della presenza di *SQL injection*. Allo stesso modo, prove di *Cross-Site Scripting* hanno dimostrato che *script* malevoli inseriti nei *form* venivano riflessi nella risposta e

interpretati dal *browser*, rendendo possibile il furto di *cookie* o la manipolazione della pagina.

Ulteriori tentativi di *command injection* hanno mostrato come l'applicazione non filtrasse adeguatamente parametri che potevano essere interpretati come comandi di sistema. Sono stati inoltre verificati scenari di *LDAP* e *XPath injection*, dove la manipolazione delle *query* consentiva di accedere a dati riservati o alterare i risultati delle ricerche. Altri *test* hanno riguardato la possibilità di includere *file* esterni non autorizzati (*Remote File Include*) e di accedere a *directory* riservate tramite sequenze di *path traversal* come `[. ./]`.

Infine, la manipolazione manuale dei *cookie* tramite strumenti di *proxy* ha evidenziato l'assenza dell'attributo *HttpOnly*, permettendo a *script* iniettati di accedere ai valori di sessione. Questi scenari hanno confermato come, in assenza di adeguati controlli, *NodeGoat* risultasse vulnerabile a molteplici forme di *injection*, coprendo gran parte dei sottopunti previsti nella categoria A3.

- **Mitigazione:** Per contrastare le vulnerabilità di *injection* rilevate in *NodeGoat*, il *WAF* è stato configurato con una serie di contromisure specifiche. In primo luogo sono state abilitate le *attack signatures* dedicate alle diverse forme di *injection*, tra cui *SQLi*, *LDAP injection*, *XPath injection*, *command injection*, *XSS* e *Remote File Include*, così da bloccare in modo proattivo i *payload* malevoli più comuni.

Parallelamente, è stato configurato il filtro sui *disallowed meta characters*, in grado di intercettare caratteri e sequenze pericolose nei parametri delle richieste (come `<`, `>`, `;`, o sequenze di *encoding*), prevenendo l'iniezione di comandi. Sono state inoltre attivate le opzioni di *HttpOnly cookie enforcement*, che hanno reso i *cookie* di sessione inaccessibili via *JavaScript*, riducendo l'impatto potenziale degli attacchi *XSS*.

Un ulteriore livello di protezione è stato ottenuto tramite il *tuning* delle regole di validazione dei parametri, imponendo vincoli su lunghezza, tipologia e caratteri ammessi, così da ridurre lo spazio di manovra per *input* malevoli. Infine, il sistema è stato configurato per rilevare e monitorare anche tentativi di evasione basati su tecniche di *encoding* o frammentazione dei *payload*, che avrebbero potuto aggirare i controlli più superficiali.

Come per le altre categorie, le regole sono state inizialmente applicate in modalità *staging*, così da analizzare i *log* e valutare l'impatto delle configurazioni, e solo successivamente portate in *enforcement*, garantendo il blocco effettivo delle richieste non conformi.

- **Validazione:** Dopo l'attivazione delle contromisure, sono stati ripetuti gli attacchi di *injection* per verificarne l'efficacia. I tentativi di *SQL injection*, *LDAP injection*, *XPath injection* e *command injection* venivano intercettati dal *WAF*, che bloccava le richieste e ne registrava il dettaglio nei *log*. Allo stesso modo, gli attacchi di *Cross-Site Scripting* non producevano più l'esecuzione di codice nel *browser*, poiché venivano rilevati dalle *signature* attive e neutralizzati prima che raggiungessero l'utente.

Le richieste che contenevano sequenze di *path traversal* o tentativi di *remote file include* venivano sistematicamente respinte con risposte di errore generate dal *firewall*. Inoltre, la configurazione di *HttpOnly* sui *cookie* di sessione impediva l'accesso ai loro valori tramite *JavaScript*, riducendo così l'impatto potenziale

degli attacchi *XSS*. Anche tecniche di evasione più sofisticate, basate su *encoding* o frammentazione dei *payload*, non risultavano più efficaci, poiché venivano rilevate e bloccate dal sistema di analisi.

Il monitoraggio tramite la sezione *Event Logs* ha confermato il corretto funzionamento delle regole configurate, mentre la *dashboard* di *OWASP Compliance* ha segnato la categoria A3 come pienamente coperta, certificando in modo oggettivo l'efficacia delle mitigazioni introdotte.

3.2.4 A4 - Insecure Design

La categoria *Insecure Design* fa riferimento a problematiche di sicurezza derivanti da una progettazione carente o assente dei meccanismi di protezione. A differenza delle vulnerabilità puramente implementative, in questo caso l'origine del rischio è riconducibile a scelte architetturali, come la mancanza di analisi dei casi d'uso, l'assenza di regole di sicurezza chiaramente definite o l'inadeguata separazione tra utenti e risorse. Nel contesto della *OWASP compliance* di F5, una *policy* risulta conforme quando implementa controlli presenti nella [Tabella 3.4](#):

Controllo	Descrizione
Predictable Resource Location	Prevenzione dell'uso di percorsi o nomi di <i>file</i> prevedibili che possano esporre risorse sensibili.
Information Leakage	Riduzione del rischio di esposizione di dettagli tecnici o informazioni riservate tramite risposte dell'applicazione.
Use Cases Analysis	Analisi dei casi d'uso dell'applicazione per identificare scenari potenzialmente vulnerabili dal punto di vista della sicurezza.
Security Rules Definitions	Definizione di regole e vincoli di sicurezza da applicare a livello applicativo per prevenire comportamenti non autorizzati.
Threat Modelling Assessments	Valutazione sistematica delle possibili minacce e dei loro impatti attraverso metodologie di threat modelling ^[g] .
Threat Modelling Testing	Test mirati per verificare che le minacce individuate tramite analisi siano effettivamente mitigabili con le contromisure adottate.
Tenant Isolation Design	Implementazione di misure di isolamento tra diversi tenant ^[g] o ambienti applicativi, per evitare interferenze o fughe di dati.
Resource Consumption Limitation	Definizione di limiti di utilizzo delle risorse (memoria, <i>CPU</i> , richieste concorrenti) per prevenire abusi e garantire la stabilità del sistema.

Tabella 3.4: Controlli previsti per A4 - Insecure Design

- **Verifica:** Durante i test condotti su *NodeGoat* sono emerse alcune vulnerabilità riconducibili a carenze di progettazione, più che a singoli errori di implementazione. Alcuni *URL* e percorsi di risorse potevano essere facilmente indovinati, a dimostrazione di una scarsa attenzione alla gestione delle *predictable resource*

location. In più casi, le risposte restituite dal *server* contenevano informazioni tecniche non necessarie all'utente, come dettagli sull'ambiente o messaggi di errore poco filtrati, segnalando la presenza di fenomeni di *information leakage*.

Un'ulteriore criticità ha riguardato l'assenza di limiti al consumo di risorse: un singolo utente poteva generare numerose richieste senza restrizioni, con il rischio di sovraccaricare l'applicazione e comprometterne la disponibilità. Nel complesso, queste problematiche hanno evidenziato come l'assenza di un'adeguata analisi dei casi d'uso e di una progettazione orientata alla sicurezza (*security by design*) renda l'applicazione vulnerabile a scenari di abuso anche senza l'impiego di tecniche di attacco particolarmente sofisticate.

- **Mitigazione:** Per ridurre i rischi derivanti da un *design* intrinsecamente insicuro, il *WAF* è stato configurato con una serie di regole volte a compensare le mancanze riscontrate. In primo luogo sono stati definiti in modo esplicito gli *Allowed URLs*, limitando l'accesso soltanto ai percorsi realmente necessari e impedendo così lo sfruttamento di risorse facilmente prevedibili. Parallelamente è stata abilitata la protezione contro l'*information leakage*, così da bloccare o filtrare le risposte che contenevano dettagli tecnici o informazioni non destinate all'utente finale.

Sono state inoltre introdotte regole di sicurezza personalizzate, calibrate sui casi d'uso specifici di *NodeGoat*, con l'obiettivo di rafforzare i controlli applicativi in punti dove l'implementazione originaria risultava carente. Per prevenire abusi di risorse, sono stati configurati profili *DoS* che limitavano il numero massimo di richieste provenienti da un singolo utente o sessione, riducendo il rischio di sovraccarico. Infine, sono state implementate regole di isolamento tra utenti, così da impedire che sessioni diverse potessero accedere a dati non propri.

Queste configurazioni, pur non eliminando le debolezze architetturali dell'applicazione, hanno rafforzato in modo significativo il livello di protezione, compensando la mancanza di un approccio nativamente orientato alla *security by design*.

- **Validazione:** Dopo l'applicazione delle regole, i *test* di attacco sono stati ripetuti per verificare l'efficacia delle contromisure adottate. I tentativi di accedere a risorse prevedibili o non autorizzate venivano sistematicamente intercettati dal *WAF*, che restituiva risposte di errore impedendo l'accesso ai contenuti. Le risposte *HTTP* non riportavano più dettagli tecnici o informazioni sensibili, a conferma dell'avvenuta attivazione della protezione contro l'*information leakage*.

Anche le prove di abuso delle risorse, condotte simulando un numero eccessivo di richieste generate dallo stesso utente, non avevano più effetto: i profili *DoS* configurati limitavano automaticamente il traffico e preservavano la stabilità del sistema. Infine, la verifica delle sessioni multiple ha mostrato che utenti distinti non potevano più interferire tra loro né accedere a dati altrui, confermando l'efficacia delle regole di isolamento introdotte.

La sezione *Event Logs* ha registrato tutte le violazioni intercettate, mentre la *dashboard* di *OWASP Compliance* ha segnalato la categoria A4 come pienamente coperta, attestando il miglioramento del livello di sicurezza complessivo.

3.2.5 A5 - Security Misconfiguration

La categoria *Security Misconfiguration* fa riferimento a tutte le vulnerabilità derivanti da configurazioni errate, incomplete o insicure di *server*, *framework* e componenti

applicativi. Spesso queste problematiche non sono legate a *bug* di programmazione, ma alla mancanza di *hardening*^[g] dei sistemi o a configurazioni permissive che lasciano esposte funzionalità non necessarie. Nel contesto della *OWASP compliance* di F5, questa categoria risulta coperta quando la *policy* implementa i controlli previsti dalla [Tabella 3.5](#):

Controllo	Descrizione
Information Leakage	Prevenzione della divulgazione di dettagli tecnici o configurazioni sensibili nelle risposte dell'applicazione.
Vulnerability Scan	Identificazione di vulnerabilità note tramite <i>scanner</i> automatici di sicurezza.
External Entity Injection Attempt	Rilevamento e blocco di tentativi di iniezione di entità esterne nei contenuti <i>XML</i> .
XML External Entity (XXE) Injection Attempt	Protezione specifica contro attacchi <i>XML External Entity (XXE)</i> ^[g] basati su contenuti <i>XML</i> manipolati.
Disallow DTDs in XML Content Profile	Blocco della definizione di <i>Document Type Definition (DTD)</i> ^[g] all'interno dei documenti <i>XML</i> , riducendo il rischio di attacchi <i>XXE</i> .
Attack Signatures	Abilitazione di firme di attacco per rilevare <i>pattern</i> noti di <i>exploit</i> ^[g] legati a configurazioni non sicure.
Server Technologies	Mascheramento o rimozione di intestazioni <i>HTTP</i> che rivelano tecnologie utilizzate dal <i>server</i> (es. versione di <i>Apache</i> ^[g] , <i>Nginx</i> ^[g] , ecc.).
Allowed Methods	Limitazione dei metodi <i>HTTP</i> accettati dal <i>server</i> , bloccando quelli non necessari (es. <i>TRACE</i> ^[g] , <i>OPTIONS</i> ^[g] , <i>PUT</i> ^[g]).
SameSite Cookie Attribute Enforcement	Applicazione dell'attributo <i>SameSite</i> ^[g] ai <i>cookie</i> , riducendo i rischi di attacchi <i>CSRF</i> .
CORS Enabled in URLs	Configurazione sicura delle regole <i>Cross-Origin Resource Sharing (CORS)</i> ^[g] , evitando l'abilitazione indiscriminata dell'accesso da domini esterni.
Clickjacking Protection URLs	Abilitazione di intestazioni di protezione (es. <i>X-Frame-Options</i> ^[g]) per impedire attacchi di <i>clickjacking</i> ^[g] .
Application Vulnerability Scanning	Scansione periodica dell'applicazione per individuare vulnerabilità introdotte da modifiche o nuove funzionalità.
Vulnerability Scanner Integration	Integrazione del <i>WAF</i> con strumenti di <i>vulnerability scanning</i> per aggiornare automaticamente le regole.
Application and System Patching	Aggiornamento regolare delle applicazioni e del sistema operativo per correggere vulnerabilità note.
Application and System Hardening	Applicazione di misure di <i>hardening</i> (disabilitazione servizi inutili, configurazioni sicure) per ridurre la superficie d'attacco.

Tabella 3.5: Controlli previsti per A5 - Security Misconfiguration

- **Verifica:** Durante la fase di verifica, le scansioni e i *test* condotti su *NodeGoat* hanno evidenziato diverse problematiche tipiche di configurazione insicura. Le risposte *HTTP* restituivano messaggi di errore e dettagli tecnici sul *server*, manifestando fenomeni di *information leakage*. Inoltre, erano abilitati metodi *HTTP* non necessari come *PUT* e *TRACE*, che avrebbero potuto essere sfruttati da un attaccante per condurre ulteriori *test* di vulnerabilità.

L'assenza di intestazioni di protezione quali *X-Frame-Options* rendeva l'applicazione vulnerabile ad attacchi di *clickjacking*, mentre i *cookie* di sessione non erano configurati con attributi di sicurezza come *SameSite* e *Secure*, facilitando possibili abusi o furti di sessione. Allo stesso modo, non vi erano controlli sulle richieste *CORS*, consentendo a domini non autorizzati di interagire con l'applicazione.

Nel complesso, queste verifiche hanno confermato che, in assenza di un'adeguata configurazione di sicurezza, *NodeGoat* risultava esposto a una vasta gamma di rischi classificabili all'interno della categoria *Security Misconfiguration*.

- **Mitigazione:** Per mitigare le vulnerabilità riconducibili a configurazioni insicure, sul *WAF* sono state attivate diverse contromisure mirate. In primo luogo, sono state abilitate le firme di attacco dedicate ai tentativi di *XML External Entity (XXE) injection*, così da bloccare l'uso di entità esterne e ridurre i rischi legati alla manipolazione di contenuti *XML*. Parallelamente è stata definita una lista ristretta di metodi *HTTP* consentiti, limitata alle sole richieste *GET* e *POST*, escludendo quelli non necessari come *TRACE* o *PUT*.

Per impedire la divulgazione di dettagli sensibili attraverso i messaggi di risposta, è stata abilitata la protezione da *information leakage*, mentre la difesa contro il *clickjacking* è stata implementata tramite l'aggiunta automatica di intestazioni *HTTP* come *X-Frame-Options*. I *cookie* di sessione sono stati rafforzati mediante l'applicazione degli attributi *SameSite* e *Secure*, riducendo i rischi di furto o abuso delle sessioni.

Un'ulteriore misura ha riguardato la gestione delle richieste *CORS*, configurata in modo da consentire esclusivamente i domini autorizzati e bloccare gli accessi provenienti da origini non fidate. Infine, sono state sfruttate le funzionalità integrate di *application vulnerability scanning* e i *log* del sistema per individuare e correggere tempestivamente configurazioni non sicure.

Queste misure, applicate inizialmente in modalità *staging* e successivamente portate in *enforcement*, hanno permesso di rafforzare in maniera significativa la postura di sicurezza dell'applicazione, compensando le debolezze derivanti da una configurazione non sicura.

- **Validazione:** Una volta applicate le contromisure, i *test* iniziali sono stati ripetuti per verificarne l'efficacia. I metodi *HTTP* non autorizzati, come *TRACE* e *PUT*, venivano respinti dal *WAF*, mentre le risposte dell'applicazione contenevano correttamente le intestazioni di protezione aggiuntive, prevenendo così potenziali attacchi di *clickjacking*. I *cookie* apparivano nei *log* con gli attributi di sicurezza *SameSite* e *Secure* applicati, confermando che non potevano più essere sfruttati in scenari di abuso di sessione.

Anche i tentativi di *XML External Entity (XXE) injection* venivano intercettati e bloccati, con la relativa registrazione nei *log* di violazione, mentre le richieste *CORS* provenienti da domini non autorizzati venivano negate in modo sistemati-

co. Tutti questi riscontri hanno dimostrato la piena efficacia delle mitigazioni introdotte.

Il monitoraggio tramite *Event Logs* ha fornito evidenze puntuali di ciascun blocco, mentre la *dashboard* di *OWASP Compliance* ha contrassegnato la categoria A5 come completamente protetta, validando così il rafforzamento delle configurazioni di sicurezza dell'applicazione.

3.2.6 A6 - Vulnerable and Outdated Components

La categoria *Vulnerable and Outdated Components* riguarda i rischi derivanti dall'utilizzo di librerie, *framework*, sistemi operativi o *middleware*^[g] che presentano vulnerabilità note e non aggiornate. Gli attaccanti possono sfruttare queste debolezze per compromettere l'applicazione, accedere a dati sensibili o eseguire codice arbitrario. Nel contesto della *OWASP compliance* di F5, la copertura della categoria A6 richiede la presenza di tutti i controlli nella [Tabella 3.6](#):

Controllo	Descrizione
Attack Signatures	Utilizzo di firme di attacco aggiornate per rilevare <i>exploit</i> noti legati a componenti vulnerabili.
Server Technologies	Mascheramento o rimozione delle intestazioni <i>HTTP</i> che rivelano la tecnologia e la versione del <i>server</i> , riducendo l'esposizione di informazioni utili agli attaccanti.
Application Vulnerability Scanner	Scansione periodica delle applicazioni per identificare librerie, <i>framework</i> o moduli con vulnerabilità note.
Vulnerability Scanner Integration	Integrazione del <i>WAF</i> con <i>scanner</i> di vulnerabilità esterni, così da aggiornare automaticamente le regole di protezione in base ai risultati.
Application and System Patching	Aggiornamento costante di applicazioni, dipendenze e sistema operativo per correggere vulnerabilità note e documentate.
Application and System Hardening	Applicazione di misure di <i>hardening</i> (rimozione di servizi inutili, configurazioni sicure, restrizioni di accesso) per ridurre la superficie d'attacco.

Tabella 3.6: Controlli previsti per A6 - Vulnerable and Outdated Components

- **Verifica:** Durante la fase di verifica, su *NodeGoat* sono state condotte scansioni di sicurezza e analisi manuali per individuare componenti non aggiornati o configurazioni deboli. I *test* hanno mostrato che le intestazioni *HTTP* restituivano informazioni dettagliate sulle tecnologie di *backend* in uso, come la versione del *runtime*^[g] *Node.js* e dei *framework* applicativi, aumentando il rischio di esposizione a *exploit* noti. Le analisi hanno inoltre evidenziato l'impiego di librerie e pacchetti obsoleti, già segnalati come vulnerabili in *database* pubblici di sicurezza, senza alcun meccanismo nativo di rilevamento o blocco.

L'assenza di controlli specifici per intercettare *exploit* rivolti a componenti non aggiornati e la mancanza di pratiche di *patching* o *hardening* hanno confermato come *NodeGoat*, in configurazione predefinita, fosse suscettibile a una vasta gamma di attacchi basati su vulnerabilità note e documentate.

- **Mitigazione:** Per ridurre i rischi associati all'utilizzo di componenti vulnerabili o non aggiornati, il *WAF* è stato configurato con una serie di misure preventive. Sono state innanzitutto abilitate le *attack signatures* specifiche per *exploit* noti legati a *framework*, librerie e *Content Management System (CMS)*^[8], così da intercettare tentativi di attacco automatizzati. Parallelamente, è stato attivato il mascheramento delle informazioni relative alle *server technologies* nelle intestazioni *HTTP*, impedendo che dettagli come versione del *runtime* o del *framework* potessero essere sfruttati da un potenziale aggressore.

La *policy* di sicurezza è stata inoltre integrata con strumenti di *application vulnerability scanning*, utilizzati per monitorare in maniera continuativa la presenza di dipendenze obsolete o vulnerabili, garantendo un riscontro immediato sulle componenti esposte a rischi noti. Infine, sono state applicate linee guida di *system hardening*, che hanno previsto la disabilitazione di servizi non necessari, la riduzione delle informazioni pubblicamente accessibili e l'adozione di configurazioni più restrittive.

Va sottolineato che il *WAF*, pur fornendo un livello di protezione avanzato, non sostituisce i processi di *patching* e aggiornamento dei *software*; tuttavia, le regole applicate hanno permesso di mitigare sensibilmente il rischio di compromissione tramite *exploit* già documentati, riducendo l'esposizione complessiva dell'applicazione.

- **Validazione:** Al termine della configurazione, sono state eseguite nuove scansioni di vulnerabilità e *test* mirati per verificare l'efficacia delle contromisure introdotte. Le intestazioni *HTTP* restituite dal *server* non riportavano più dettagli sensibili relativi a versioni o tecnologie utilizzate, a conferma che il mascheramento delle *server technologies* era attivo e funzionante. I tentativi di sfruttare *exploit* già noti, simulati tramite *payload* specifici, venivano sistematicamente intercettati dalle firme del *WAF* e registrati nei *log* come violazioni bloccate.

Il monitoraggio tramite la sezione *Event Logs* ha permesso di confermare ogni intervento del sistema, mentre la *dashboard* di *OWASP Compliance* ha classificato la categoria A6 come *Protected*, certificando in maniera oggettiva la copertura ottenuta e l'efficacia complessiva delle mitigazioni applicate.

3.2.7 A7 - Identification and Authentication Failures

La categoria *Identification and Authentication Failures* comprende le vulnerabilità legate a meccanismi di autenticazione e gestione delle sessioni non adeguatamente sicuri. Errori nella validazione delle credenziali, nella protezione dei *cookie* di sessione o nell'implementazione dei controlli di *login* possono consentire ad un attaccante di impersonare altri utenti o di accedere a dati riservati. Nel contesto della *OWASP compliance* di F5, la categoria A7 è coperta se la *policy* prevede i controlli in [Tabella 3.7](#):

Controllo	Descrizione
Authentication / Authorization Attacks	Rilevamento e blocco di tentativi di aggirare i meccanismi di autenticazione o di accedere a risorse senza autorizzazione.
ASM Cookie Tampering Protection	Protezione contro la manipolazione dei <i>cookie</i> generati dal modulo <i>ASM</i> , prevenendo alterazioni dei valori di sessione.
Session Hijacking Protection	Difesa contro il furto o il riutilizzo di sessioni attive da parte di attaccanti, ad esempio tramite <i>cookie</i> rubati.
Brute Force Protection	Limitazione dei tentativi di accesso consecutivi non riusciti, per bloccare attacchi di forza bruta sulle credenziali.
Credentials Stuffing Protection	Rilevamento e blocco di accessi multipli basati su credenziali compromesse provenienti da <i>data breach</i> .
Login Enforcement	Obbligo di passare attraverso la pagina di autenticazione prevista, impedendo accessi diretti a risorse senza sessione valida.

Tabella 3.7: Controlli previsti per A7 - Identification and Authentication Failures

- **Verifica:** Durante i *test* di verifica condotti su *NodeGoat*, sono stati simulati diversi scenari di attacco rivolti ai meccanismi di autenticazione e gestione delle sessioni. Le prove di *brute force* hanno evidenziato come l'applicazione consentisse un numero illimitato di tentativi di accesso, senza alcuna misura di blocco o ritardo, rendendo possibile indovinare le credenziali tramite attacchi automatizzati. In parallelo, sono stati effettuati *test* di *credential stuffing*, utilizzando liste di credenziali compromesse, che non venivano rilevate né bloccate dal sistema.

Ulteriori analisi hanno riguardato la manipolazione manuale dei *cookie* di sessione, che ha dimostrato l'assenza di controlli contro il *cookie tampering*: modificando i valori era possibile simulare un innalzamento dei privilegi. Infine, simulazioni di *session hijacking*^[g], condotte catturando *cookie* non protetti durante la navigazione, hanno confermato che la sessione poteva essere riutilizzata da un attaccante senza alcun meccanismo di protezione attivo.

Nel complesso, questi scenari hanno mostrato che, in assenza di adeguate contromisure, *NodeGoat* risultava vulnerabile a diverse tipologie di abusi legati all'autenticazione e alla gestione delle sessioni.

- **Mitigazione:** Per contrastare le vulnerabilità riscontrate nei meccanismi di autenticazione e gestione delle sessioni, sul *WAF* sono state applicate diverse contromisure mirate. È stato innanzitutto configurato un profilo di *Brute Force Prevention*, che applica regole di *rate limiting* e blocca automaticamente gli indirizzi *IP* responsabili di un numero eccessivo di tentativi falliti di accesso. In parallelo è stata attivata la protezione contro il *Credential Stuffing*, in grado di riconoscere e filtrare richieste sospette basate su insiemi noti di credenziali compromesse.

Per quanto riguarda la protezione delle sessioni, è stata abilitata la funzionalità di *ASM cookie tampering protection*, che impedisce la modifica arbitraria dei *cookie* generati dal sistema, garantendo l'integrità delle informazioni di sessione. A ciò si affianca l'*enforcement* delle regole contro il *session hijacking*, ottenuto

monitorando costantemente la validità delle sessioni utente e rilevando eventuali anomalie. Infine, è stato attivato il modulo di *Login Enforcement*, che obbliga gli utenti a passare attraverso la pagina di autenticazione prevista, impedendo l'accesso diretto a risorse protette senza una sessione valida.

Queste configurazioni, applicate inizialmente in modalità *staging* e successivamente in *blocking*, hanno rafforzato in maniera significativa la sicurezza del processo di autenticazione, riducendo drasticamente le possibilità di abuso.

- **Validazione:** Dopo l'attivazione delle contromisure, gli scenari di attacco sono stati ripetuti per verificare l'efficacia delle configurazioni introdotte. I tentativi di *brute force* sull'interfaccia di *login* venivano automaticamente interrotti al superamento della soglia definita, con la generazione del relativo *log* di violazione. Analogamente, le prove di *credential stuffing* producevano allarmi e blocchi da parte del *WAF*, impedendo l'uso di credenziali compromesse.

Le manipolazioni dei *cookie* di sessione, effettuate manualmente tramite *proxy*, venivano immediatamente intercettate e respinte grazie alla protezione contro il *cookie tampering*, mentre i tentativi di *session hijacking* non avevano più effetto grazie ai controlli di integrità applicati alle sessioni utente. Infine, ogni accesso diretto a risorse protette senza *login* veniva negato dal meccanismo di *Login Enforcement*, che obbligava l'utente a passare per la pagina di autenticazione prevista.

Il monitoraggio tramite la sezione *Event Logs* ha confermato in ciascun caso l'intercettazione e il blocco degli attacchi, mentre la *dashboard* di *OWASP Compliance* ha mostrato la categoria A7 come pienamente coperta, attestando il rafforzamento dei meccanismi di identificazione e autenticazione.

3.2.8 A8 - Software and Data Integrity Failures

La categoria *Software and Data Integrity Failures* comprende le vulnerabilità che si manifestano quando il *software* o i dati non vengono adeguatamente protetti nella loro integrità. In assenza di controlli robusti, un attaccante potrebbe modificare componenti, pacchetti o configurazioni, compromettendo il corretto funzionamento dell'applicazione. Questo tipo di rischio si estende anche alla *supply chain*^[8] del *software* e ai processi di distribuzione continua (*CI/CD*). Nel contesto della *OWASP compliance* di F5, la categoria A8 è considerata coperta se la *policy* include tutti i controlli mostrati nella [Tabella 3.8](#):

Controllo	Descrizione
Buffer Overflow	Prevenzione di <i>input</i> eccessivi progettati per sovrascrivere la memoria del <i>server</i> e compromettere l'integrità del sistema.
Command Execution	Rilevamento e blocco di <i>input</i> che mirano a eseguire comandi arbitrari sul sistema bersaglio.
Denial of Service	Limitazione di richieste malevole o sovraccarichi di <i>input</i> che compromettono la disponibilità dell'applicazione.
Server-Side Code Injection	Protezione contro l'inserimento ed esecuzione di codice non autorizzato lato <i>server</i> .
Enforced Cookies	Applicazione di regole di integrità e sicurezza sui <i>cookie</i> (es. validazione forzata, attributi protettivi) per impedire manipolazioni.
Use Trusted Repositories	Adozione esclusiva di repository ^[8] affidabili per il <i>download</i> di librerie e pacchetti, riducendo il rischio di dipendenze compromesse.
Packages Digitally Signed and Verified	Utilizzo di pacchetti firmati digitalmente e verifica delle firme per garantirne autenticità e integrità.
Secure CI/CD Pipeline	Implementazione di <i>pipeline</i> di integrazione e distribuzione continua con controlli di sicurezza, scansioni e validazioni automatiche.
Component Inventory Tool	Utilizzo di strumenti di inventario delle componenti <i>software</i> per tracciare versioni, dipendenze e vulnerabilità associate.

Tabella 3.8: Controlli previsti per A8 - Software and Data Integrity Failures

- **Verifica:** Durante la fase di verifica condotta su *NodeGoat*, sono emerse diverse criticità riconducibili alla categoria *Software and Data Integrity Failures*. In particolare, attraverso l'inserimento di *input* malevoli nei campi disponibili, è stato possibile eseguire forme di *injection* lato *server*, che consentivano l'esecuzione non autorizzata di codice. Allo stesso modo, tentativi di [command execution](#)^[8] hanno dimostrato che l'applicazione non validava correttamente i parametri ricevuti, permettendo l'invio di comandi arbitrari al *backend*.

Un'ulteriore vulnerabilità ha riguardato l'assenza di controlli di integrità sui *cookie*, che potevano essere manipolati e riutilizzati senza alcuna verifica, compromettendo la sicurezza delle sessioni. Infine, è stata rilevata la mancanza di meccanismi a livello applicativo per garantire l'utilizzo esclusivo di componenti affidabili o tracciati: l'assenza di un inventario delle dipendenze e di controlli sulle fonti riduceva la capacità di individuare pacchetti non sicuri o compromessi.

Nel complesso, questi scenari hanno mostrato come *NodeGoat*, nella sua configurazione di *default*, risultasse esposto a rischi per l'integrità sia del *software* sia dei dati trattati.

- **Mitigazione:** Per contrastare le vulnerabilità legate all'integrità del *software* e dei dati, il *WAF* è stato configurato con un insieme di contromisure mirate. Sono state innanzitutto abilitate le *attack signatures* dedicate al rilevamento di *exploit*

comuni, come i tentativi di [buffer overflow](#)^[g], *command execution* e *server-side code injection*, così da bloccare preventivamente i *payload* malevoli più diffusi. A queste è stata affiancata la configurazione di profili di protezione contro il *Denial of Service*, utili a limitare l'impatto di richieste massive o di consumo eccessivo di risorse.

Un altro intervento ha riguardato la gestione dei *cookie* di sessione: mediante le opzioni di *cookie enforcement* sono stati imposti attributi di sicurezza e controlli di coerenza, rendendo impossibile la manipolazione arbitraria dei valori memorizzati. Infine, la configurazione delle *policy* di sicurezza ha richiamato le *best practice* di gestione della *supply chain*, promuovendo l'utilizzo di *repository* affidabili, l'aggiornamento costante dei componenti e la tracciabilità delle dipendenze tramite strumenti di monitoraggio.

Queste misure hanno permesso al *WAF* di compensare parte delle debolezze intrinseche dell'applicazione, riducendo sensibilmente il rischio di compromissione dell'integrità del *software* e dei dati trattati.

- **Validazione:** Nella fase di validazione sono stati ripetuti gli attacchi simulati per verificare l'efficacia delle configurazioni introdotte. I tentativi di *injection* lato *server* e di *command execution* venivano sistematicamente intercettati e bloccati dalle firme di attacco attive, con la registrazione puntuale delle violazioni nei *log*. Le manipolazioni sui *cookie* di sessione non producevano più alcun effetto: il sistema applicava infatti i controlli di coerenza e gli attributi di sicurezza previsti dall'*enforcement*, garantendone l'integrità.

Anche i *test* mirati a generare un carico eccessivo di richieste non riuscivano a compromettere la disponibilità dell'applicazione, poiché i profili *DoS* configurati limitavano automaticamente il traffico sospetto e preservavano le risorse. L'insieme di queste evidenze è stato confermato dalla *dashboard* di *OWASP Compliance*, che ha marcato la categoria A8 come pienamente protetta, attestando in maniera oggettiva l'efficacia delle contromisure adottate.

3.2.9 A9 - Security Logging and Monitoring Failures

La categoria *Security Logging and Monitoring Failures* riguarda la mancanza o l'insufficienza dei meccanismi di registrazione e monitoraggio degli eventi di sicurezza. Senza *log* accurati e un sistema di allerta tempestivo, le violazioni potrebbero passare inosservate, impedendo una risposta rapida ed efficace. Nel contesto della *OWASP compliance* di F5, la categoria A9 è considerata coperta quando la *policy* garantisce l'esecuzione dei controlli in [Tabella 3.9](#):

Controllo	Descrizione
Log Illegal Requests	Registrazione sistematica delle richieste malevole o non conformi, con indicazione del tipo di violazione e dell'azione intrapresa.
Remote Logging	Inoltre dei <i>log</i> a un sistema centralizzato o a un Security Information and Event Management (SIEM) ^[g] , per consentire il monitoraggio continuo e la correlazione degli eventi di sicurezza.

Tabella 3.9: Controlli previsti per A9 - Security Logging and Monitoring Failures

- **Verifica:** Durante la fase di verifica, su *NodeGoat* sono stati condotti diversi attacchi simulati, tra cui tentativi di *injection*, *brute force* e *XSS*, al fine di valutare l'efficacia del sistema di registrazione degli eventi. L'analisi dei *log* ha mostrato che molte richieste malevole non venivano tracciate in maniera completa: i dettagli registrati risultavano spesso insufficienti per identificare con precisione la natura dell'attacco o l'utente coinvolto. Inoltre, i *log* locali apparivano frammentati e difficili da correlare, ostacolando l'analisi di scenari più complessi che richiedono la ricostruzione di una sequenza di eventi.

Queste mancanze rendevano complessa l'individuazione tempestiva di un attacco in corso e avrebbero potuto ritardare la capacità di risposta a un incidente di sicurezza.

- **Mitigazione:** Per colmare le lacune riscontrate in termini di *logging* e monitoraggio, sul *WAF* sono state applicate configurazioni volte a garantire una tracciabilità completa degli eventi. È stato abilitato il *detailed logging* per tutte le richieste, incluse quelle malevole, in modo da registrare parametri, *URL*, indirizzi *IP* e le firme di attacco che hanno generato la violazione. Questo ha reso possibile un'analisi più approfondita del traffico e la ricostruzione accurata degli scenari di attacco. Inoltre, l'interfaccia *Event Logs* della piattaforma F5 è stata utilizzata per monitorare in tempo reale le richieste bloccate o sospette, facilitando il rilevamento immediato di attività anomale.

Queste misure hanno permesso di rafforzare significativamente le capacità di *logging* e monitoraggio, riducendo i tempi di rilevazione di un attacco e migliorando la possibilità di risposta a un incidente di sicurezza.

- **Validazione:** Dopo l'attivazione delle nuove configurazioni, gli attacchi iniziali sono stati ripetuti per verificare l'efficacia del sistema di *logging*. Ogni richiesta malevola veniva correttamente registrata nei *log* locali, con un livello di dettaglio sufficiente a consentire un'analisi forense completa, comprensiva di parametri, indirizzi *IP* e firme di attacco coinvolte.

Il monitoraggio in tempo reale tramite la console *Event Logs* di F5 confermava la rilevazione e il blocco di ciascuna violazione, mentre la *dashboard* di *OWASP Compliance* ha marcato la categoria A9 come pienamente coperta, attestando la raggiunta visibilità sugli eventi e la capacità di risposta tempestiva agli incidenti.

3.2.10 A10 - Server-Side Request Forgery (SSRF)

La categoria *Server-Side Request Forgery (SSRF)* si riferisce a vulnerabilità in cui un'applicazione *web*, se opportunamente manipolata, viene indotta a eseguire richieste *HTTP* verso destinazioni arbitrarie, interne o esterne. In questo modo un attaccante può sfruttare l'applicazione come *proxy* per accedere a sistemi non esposti, leggere dati riservati o lanciare attacchi verso servizi interni. Nel contesto della *OWASP compliance* di F5, la categoria A10 è considerata coperta se la *policy* prevede i controlli mostrati nella [Tabella 3.10](#):

Controllo	Descrizione
SSRF Attack Signatures	Firme dedicate al rilevamento di richieste malevole inviate dal <i>server</i> verso risorse interne o esterne non autorizzate.
SSRF Hosts	Definizione di liste di <i>host</i> ^[g] o domini consentiti, con blocco automatico delle richieste verso destinazioni non autorizzate.
URI or Auto-Detected Parameters	Analisi e validazione dei parametri <i>Uniform Resource Identifier (URI)</i> ^[g] o dei parametri rilevati automaticamente, per impedire che vengano sfruttati in attacchi <i>Server-Side Request Forgery (SSRF)</i> ^[g] .

Tabella 3.10: Controlli previsti per A10 - Server-Side Request Forgery (SSRF)

- **Verifica:** Durante la fase di verifica su *NodeGoat* sono stati condotti *test* mirati a valutare la presenza di vulnerabilità di tipo *Server-Side Request Forgery (SSRF)*. Manipolando i parametri che permettevano di specificare *URL* remoti, è stato possibile forzare l'applicazione a generare richieste verso indirizzi *IP* interni alla rete, come quelli appartenenti al *range* 10.1.x.x, senza alcuna restrizione.

Analogamente, modificando i parametri delle *URL*, l'applicazione accettava di effettuare connessioni verso domini arbitrari scelti dall'attaccante, inclusi *host* esterni non autorizzati. Non erano inoltre presenti controlli in grado di impedire richieste verso risorse locali o servizi sensibili, come *endpoint*^[g] interni all'infrastruttura.

Questi scenari hanno confermato che *NodeGoat*, in assenza di adeguate contromisure, risultava vulnerabile a tipici attacchi *SSRF*, con il rischio di sfruttare il *server* come *proxy* per raggiungere risorse interne o esfiltrare dati da servizi altrimenti inaccessibili.

- **Mitigazione:** Per ridurre il rischio di *Server-Side Request Forgery*, sul *WAF* sono state configurate diverse contromisure specifiche. In primo luogo sono state abilitate le *SSRF attack signatures*, progettate per intercettare *pattern* noti di attacco nei parametri e negli *URL*, così da bloccare preventivamente richieste potenzialmente malevole. Parallelamente è stata definita una lista di destinazioni consentite (*allowed hosts*), limitando le connessioni *server-side* a un insieme ristretto di indirizzi sicuri e bloccando qualsiasi tentativo di accesso verso *host* interni o domini non autorizzati.

A queste misure si è aggiunta l'applicazione di controlli stringenti sui parametri *URI*, in modo da impedire che valori arbitrari potessero essere utilizzati per generare richieste verso risorse esterne o interne non previste. In questo modo è stata ridotta drasticamente la possibilità che l'applicazione potesse fungere da *proxy* involontario, proteggendo l'infrastruttura sia da accessi non autorizzati a sistemi interni, sia da esfiltrazioni di dati verso l'esterno.

- **Validazione:** Al termine della configurazione, i *test* di attacco sono stati ripetuti per verificare l'efficacia delle contromisure applicate. I tentativi di inviare richieste *SSRF* verso *host* interni alla rete venivano sistematicamente intercettati e bloccati dal *WAF*, che generava *log* dettagliati con indicazione dell'origine della richiesta e della firma attivata. Analogamente, le connessioni verso *URL* non inclusi nella

lista degli *allowed hosts* venivano immediatamente respinte, impedendo così l'uso dell'applicazione come *proxy* per contattare destinazioni non autorizzate.

La verifica tramite la sezione *Event Logs* ha confermato il corretto funzionamento delle regole, mentre la *dashboard* di *OWASP Compliance* ha marcato la categoria A10 come completamente protetta, permettendo di raggiungere il punteggio massimo di compliance (10/10) e completando il percorso di messa in sicurezza dell'applicazione.

3.3 Risultato finale e osservazioni

Al termine delle attività di configurazione e *tuning*, il *WAF* di F5 è stato in grado di garantire la protezione completa di *NodeGoat* rispetto a tutte le dieci categorie individuate dallo *standard OWASP Top 10*. La *dashboard* di *OWASP Compliance* ha infatti riportato il punteggio massimo di 10/10, attestando che per ciascuna categoria erano stati attivati i controlli richiesti. Questo risultato, raggiunto progressivamente attraverso un approccio iterativo di verifica, mitigazione e validazione, rappresenta una dimostrazione concreta dell'efficacia delle *policy* configurate e conferma come un *WAF* correttamente impostato possa elevare significativamente il livello di sicurezza anche in un'applicazione volutamente vulnerabile come *NodeGoat*.

L'analisi delle tecniche adottate ha mostrato che non tutte le funzionalità del *WAF* hanno avuto lo stesso impatto. Le *attack signatures* si sono rivelate decisive contro le vulnerabilità più comuni (*SQLi*, *XSS*, *Command Injection*, *SSRF*), mentre funzioni come *Login Enforcement* e *Brute Force Prevention* hanno rafforzato la protezione dei sistemi di autenticazione. L'*enforcement* degli attributi di sicurezza dei *cookie* ha migliorato la gestione delle sessioni, mentre strumenti complementari come *Data Guard* e il mascheramento nei *log* hanno ridotto l'esposizione di informazioni sensibili. Un ruolo trasversale è stato svolto dal monitoraggio continuo tramite *log* e *dashboard*, che ha permesso di validare ogni modifica e ottimizzare progressivamente le configurazioni.

Nonostante il pieno raggiungimento della *compliance*, sono emerse alcune limitazioni. Le configurazioni sono state testate in un ambiente controllato e potrebbero richiedere adattamenti significativi in scenari reali e dinamici. L'attivazione iniziale di regole restrittive ha generato falsi positivi, richiedendo un lavoro costante di *tuning*, e va ricordato che un *WAF* non sostituisce processi di aggiornamento e gestione delle dipendenze *software*.

Capitolo 4

Conclusioni

Questo capitolo raccoglie le considerazioni conclusive relative al progetto di *stage* e alla stesura della tesi. Dopo un riepilogo del lavoro svolto e dei risultati raggiunti, vengono presentate alcune riflessioni di carattere tecnico e personale, evidenziando i punti di forza e le criticità riscontrate.

4.1 Riepilogo del lavoro svolto

Il percorso è iniziato con una fase di formazione della durata di due settimane, durante la quale sono stati svolti venti laboratori pratici basati sull'applicazione *OWASP Juice Shop*. Questa attività ha permesso di acquisire familiarità con l'ambiente virtuale predisposto, comprendere le funzionalità principali del *firewall* applicativo e sperimentare le prime tecniche di attacco e difesa.

La seconda fase, svolta in autonomia per le successive cinque settimane, si è concentrata sull'applicazione *OWASP NodeGoat*, scelta come *target* principale del progetto. Le attività hanno seguito uno schema metodologico iterativo articolato in tre fasi: *verifica* delle vulnerabilità, *mitigazione* tramite configurazione del *WAF* e *validazione* delle contromisure adottate.

Per ciascuna delle dieci categorie dello *standard OWASP Top 10* sono stati condotti *test* mirati per simulare attacchi reali, analizzare le debolezze dell'applicazione e applicare le configurazioni necessarie a garantire la protezione. Tra le funzionalità più rilevanti utilizzate si annoverano l'attivazione delle *attack signatures*, i meccanismi di *login enforcement* e *brute force prevention*, la protezione avanzata dei *cookie* e delle sessioni, l'uso di *Data Guard* e *masking*, nonché l'integrazione con *IP Intelligence* e profili *DoS*.

Il risultato finale è stato il raggiungimento del punteggio massimo di *compliance*, a dimostrazione che la configurazione realizzata è stata in grado di coprire tutte le dieci categorie di rischio individuate da *OWASP*. Questo traguardo ha rappresentato non solo la validazione tecnica delle attività svolte, ma anche il coronamento dell'obiettivo formativo previsto per lo *stage*.

4.2 Considerazioni tecniche

Dal punto di vista dell'efficacia, il *firewall* applicativo ha dimostrato di essere uno strumento in grado di intercettare e bloccare una vasta gamma di attacchi riconducibili

allo *standard OWASP Top 10*. Le *attack signatures* hanno garantito una copertura immediata contro vulnerabilità note, mentre funzionalità più avanzate come *Login Enforcement*, *Brute Force Prevention*, *IP Intelligence* e i profili *DoS* hanno consentito di estendere la protezione a scenari più complessi e sofisticati.

Dal punto di vista dell'usabilità, l'interfaccia di configurazione si è rivelata intuitiva nelle sue funzioni di base, ma complessa per quanto riguarda il *tuning* delle *policy*. L'attivazione indiscriminata di tutte le firme o di controlli troppo restrittivi ha portato inizialmente a diversi falsi positivi, rendendo necessario un lavoro accurato di ottimizzazione. Questo aspetto conferma che l'uso efficace di un *WAF* non può prescindere da una conoscenza approfondita sia delle vulnerabilità applicative sia del contesto operativo in cui viene inserito.

In sintesi, il progetto ha evidenziato che le tecnologie adottate sono estremamente efficaci se correttamente configurate e monitorate, ma richiedono un costante lavoro di manutenzione, aggiornamento e affinamento per garantire la massima protezione in contesti reali.

4.3 Considerazioni personali e professionali

Lo *stage* ha rappresentato un'esperienza significativa sia dal punto di vista formativo che professionale. L'attività svolta mi ha permesso di approfondire concretamente tematiche di sicurezza applicativa, applicando conoscenze teoriche a scenari pratici e confrontandomi con strumenti adottati in ambito aziendale.

Dal punto di vista personale, l'esperienza è stata stimolante: poter lavorare su un'applicazione vulnerabile come *NodeGoat* e configurare un *WAF* ha reso tangibili concetti che, fino a quel momento, avevo affrontato principalmente in ambito personale. In particolare, la possibilità di osservare direttamente come un attacco venga intercettato e bloccato dal sistema ha consolidato la mia comprensione delle vulnerabilità più comuni e delle relative contromisure.

A livello professionale, lo *stage* ha offerto l'opportunità di sperimentare un contesto aziendale strutturato, caratterizzato da obiettivi chiari, scadenze definite e costante confronto con il tutor aziendale. Questo ha permesso non solo di sviluppare competenze tecniche, ma anche di acquisire capacità organizzative e di gestione del lavoro in autonomia.

Tra gli aspetti che ho maggiormente apprezzato vi è stata la varietà delle attività: dai *test* di attacco, alla configurazione delle *policy*, fino alla validazione tramite *dashboard* e *log*. La natura iterativa del lavoro ha reso evidente quanto la sicurezza sia un processo dinamico e mai completamente concluso.

4.4 Sviluppi futuri e possibili miglioramenti

Un primo miglioramento riguarda l'integrazione con *pipeline* di sviluppo e distribuzione (*CI/CD*): automatizzare i *test* di sicurezza e l'aggiornamento delle *policy* del *WAF* consentirebbe di ridurre i tempi di reazione alle nuove minacce e di garantire una protezione costante durante l'intero ciclo di vita del *software*.

Un altro aspetto di rilievo è l'adozione estensiva di *feed* di *threat intelligence*^[g]: sebbene sia già stato utilizzato il modulo di *IP Intelligence*, l'integrazione con ulteriori fonti di reputazione e indicatori di compromissione potrebbe rafforzare la capacità del sistema di individuare attacchi sofisticati o mirati.

Dal punto di vista tecnico, sarebbe utile approfondire l'integrazione con sistemi di *logging* e *monitoring* avanzati, come i *SIEM*, per migliorare la correlazione tra eventi di sicurezza e fornire un quadro più completo delle minacce. In un ambiente aziendale complesso, la sola visibilità fornita dal *WAF* potrebbe non essere sufficiente per individuare attacchi distribuiti o correlati ad altri livelli dell'infrastruttura.

Acronimi e abbreviazioni

API [Application Programming Interface](#). 3, 40

ASM [Application Security Manager](#). 6, 40

CD [Continuous Delivery](#). 6, 41

CI [Continuous Integration](#). 6, 41

CMS [Content Management System](#). 27, 41

CORS [Cross-Origin Resource Sharing](#). 24, 42

CSRF [Cross-Site Request Forgery](#). 9, 42

DoS [Denial of Service](#). 5, 42

DTD [Document Type Definition](#). 24, 42

HTTP [HyperText Transfer Protocol](#). 3, 43

HTTPS [HyperText Transfer Protocol Secure](#). 3, 44

IP [Internet Protocol](#). 8, 44

JSON [JavaScript Object Notation](#). 3, 44

LDAP [Lightweight Directory Access Protocol](#). 20, 44

OSI [Open Systems Interconnection](#). 4, 45

OWASP [Open Worldwide Application Security Project](#). 1, 45

PWA [Progressive Web Application](#). 3, 46

SIEM [Security Information and Event Management](#). 31, 47

SPA [Single Page Application](#). 3, 47

SQL [Structured Query Language](#). 6, 47

SQLi [SQL injection](#). 8, 47

SSRF [Server-Side Request Forgery](#). 33, 47

TLS [Transport Layer Security](#). 3, 48

URI [Uniform Resource Identifier](#). 33, 48

URL [Uniform Resource Locator](#). 5, 48

WAF [Web Application Firewall](#). 1, 48

XML [eXtensible Markup Language](#). 3, 49

XPath [XML Path Language](#). 20, 49

XSS [Cross-Site Scripting](#). 8, 49

XXE [XML External Entity](#). 24, 49

Glossario

Access Control Insieme di meccanismi e politiche che regolano l'accesso alle risorse di un sistema informatico, assicurando che solo gli utenti autorizzati possano eseguire determinate operazioni. [9](#)

Apache Uno dei *server web* più diffusi e utilizzati al mondo. Supporta numerosi moduli e funzionalità per la gestione di siti *web* e applicazioni, ed è spesso impiegato in ambienti *Linux*. [24](#)

API Insieme di regole, protocolli e strumenti che consentono a *software* diversi di comunicare tra loro. Una *API* definisce i metodi e i formati di interazione, facilitando l'integrazione tra sistemi e applicazioni. Nel contesto *web*, le *API* vengono spesso esposte tramite protocolli come *HTTP* e formati come *JSON* o *XML*. [38](#)

ASM Modulo della piattaforma *BIG-IP* di F5 progettato per offrire protezione a livello applicativo. *ASM* funge da *WAF*, fornendo funzionalità avanzate come firme di attacco, *IP Intelligence*, protezione da *XSS*, *SQLi* e gestione delle *policy* per difendere le *web application* da minacce note e sconosciute. [38](#)

Auditing Processo di revisione e analisi sistematica delle attività e dei *log* di un sistema informatico, finalizzato a verificare la conformità a *policy*, normative o requisiti di sicurezza. L'*auditing* consente di rilevare anomalie, incidenti di sicurezza e di mantenere la tracciabilità delle operazioni. [6](#)

BIG-IP Piattaforma *hardware* e *software* sviluppata da F5 che offre funzionalità avanzate di bilanciamento del carico (*load balancing*), sicurezza applicativa, gestione del traffico e ottimizzazione delle prestazioni delle applicazioni *web*. [5](#)

Bot Programma automatico che effettua operazioni su *Internet*. I *bot* possono essere usati per scopi legittimi (ad esempio motori di ricerca) o malevoli (attacchi automatizzati, *spam*). Un *WAF* spesso implementa meccanismi di difesa contro il traffico generato da *bot* dannosi. [14](#)

Brute Force Attacco che tenta di ottenere l'accesso a un sistema o servizio provando sistematicamente tutte le combinazioni possibili di credenziali (*username* e *password*) o chiavi di cifratura, fino a trovare quella corretta. Le moderne difese, come i *WAF*, implementano meccanismi per rilevare e bloccare tali tentativi. [7](#)

Buffer Overflow Vulnerabilità di programmazione che si verifica quando un programma scrive più dati di quanti ne possa contenere un *buffer*, andando a sovrascrivere aree di memoria adiacenti. Un attaccante può sfruttare un *buffer overflow* per

alterare il flusso di esecuzione di un'applicazione, causare *crash* o eseguire codice arbitrario sul sistema vulnerabile. 31

Burp Suite *Suite* integrata di strumenti per *test* di sicurezza delle applicazioni *web*. Permette di eseguire analisi del traffico *HTTP/HTTPS*, attacchi automatizzati, manipolazione di richieste e molto altro. 7

CD Pratica di sviluppo *software* che estende la *CI* automatizzando l'intero processo di rilascio, garantendo che il codice possa essere distribuito in produzione in qualsiasi momento in modo sicuro e affidabile. 38

CI Pratica di sviluppo *software* che prevede l'integrazione frequente e automatizzata del codice sorgente in un *repository* condiviso, seguita dall'esecuzione di *test* automatici per individuare rapidamente errori e problemi di integrazione. 38

Clickjacking Tecnica di attacco in cui un utente viene indotto a cliccare su elementi nascosti o mascherati di una pagina *web*, compiendo azioni non intenzionali. Il *clickjacking* sfrutta la sovrapposizione di contenuti tramite *frame* o *stylesheet*, spesso in combinazione con l'assenza di *header* di sicurezza come *X-Frame-Options*. 24

Client Dispositivo o applicazione che richiede l'accesso a risorse o servizi offerti da un *server*. Nel modello *client-server*, il *client* invia richieste attraverso la rete e riceve risposte contenenti i dati o i servizi richiesti. 3

CMS Piattaforma *software* che consente di creare, gestire e modificare contenuti digitali, tipicamente per siti *web*, senza richiedere competenze avanzate di programmazione. Un *CMS* offre strumenti per organizzare pagine, articoli, media e utenti, facilitando la manutenzione e l'aggiornamento dei contenuti *online*. 38

Command Execution Tipologia di vulnerabilità che consente a un attaccante di eseguire comandi arbitrari sul sistema *target*. Può essere suddivisa in *Remote Command Execution (RCE)*, quando l'attacco viene condotto da remoto, e *Local Command Execution (LCE)*, quando viene sfruttato un accesso locale. Questo tipo di attacco può compromettere gravemente la sicurezza di un sistema, permettendo il furto di dati, l'installazione di *malware* o il controllo completo del dispositivo. 30

Command Injection Vulnerabilità di sicurezza che consente a un attaccante di inserire e far eseguire comandi arbitrari sul sistema operativo sottostante, sfruttando un'applicazione che non valida o filtra correttamente gli *input* dell'utente. Gli attacchi di *command injection* possono compromettere la riservatezza, l'integrità e la disponibilità del sistema. 8

Container Tecnologia di virtualizzazione a livello di sistema operativo che consente di eseguire applicazioni e le loro dipendenze in ambienti isolati e portabili. I *container* condividono il *kernel* del sistema operativo ma mantengono un *file system*, librerie e configurazioni indipendenti, garantendo coerenza e ripetibilità negli ambienti di sviluppo, *test* e produzione. 10

Cookie Piccolo *file* di testo che un *server* invia al *client* (solitamente un *browser*) per memorizzare informazioni di stato o preferenze dell'utente. I *cookie* sono utilizzati per sessioni di autenticazione, tracciamento delle attività di navigazione e personalizzazione dei contenuti. 5

CORS Meccanismo di sicurezza del *web* che consente o limita le richieste *HTTP* provenienti da domini diversi rispetto a quello dell'applicazione. È gestito tramite specifici *header* inviati dal *server*, usato per controllare le risorse accessibili tra origini differenti. 38

Credential Stuffing Tecnica di attacco in cui un attore malevolo utilizza in modo automatizzato combinazioni di *username* e *password* rubate da precedenti violazioni di dati, tentando di accedere ad altri servizi o applicazioni in cui l'utente potrebbe aver riutilizzato le stesse credenziali. 5

CSRF Vulnerabilità delle applicazioni *web* in cui un attaccante induce un utente autenticato a eseguire azioni indesiderate su un'applicazione fidata, sfruttando la fiducia del sito verso il *browser* dell'utente. 38

Data Guard Tecnologia di protezione e alta disponibilità dei dati sviluppata da *Oracle*, progettata per creare, mantenere e gestire copie di *database* sincronizzate (*standby*) al fine di garantire il recupero rapido in caso di guasti o interruzioni. Il *Data Guard* può essere utilizzato per *disaster recovery*, bilanciamento del carico di lettura e protezione contro la perdita di dati. 14

Docker Piattaforma *software* che consente di sviluppare, eseguire e gestire applicazioni in *container* leggeri e portabili. I *container Docker* permettono di isolare l'applicazione e le sue dipendenze dal sistema operativo sottostante. 10

DoS Attacco informatico finalizzato a rendere indisponibile un servizio, una risorsa di rete o un'intera infrastruttura, sovraccaricando i *server* o saturando la banda con richieste malevole o massive. 38

DTD Insieme di regole che definisce la struttura e gli elementi consentiti in un documento *XML*. La *DTD* può includere entità interne ed esterne, che, se non gestite correttamente, possono introdurre vulnerabilità come *XXE*. 38

Encoding Processo di trasformazione di dati da un formato a un altro secondo regole specifiche, tipicamente per garantire compatibilità o trasmissione sicura. Nel contesto della sicurezza *web*, l'*encoding* viene utilizzato per prevenire vulnerabilità come *XSS*, codificando caratteri speciali. 20

Endpoint Punto di accesso a una risorsa o a un servizio disponibile su una rete. Nel contesto di questo progetto, rappresenta un *URL* specifico al quale un *client* può inviare richieste per eseguire operazioni su dati o funzionalità esposte da un *server*. 33

End-to-End Approccio o modalità di progettazione, implementazione e gestione in cui un processo, un servizio o una comunicazione viene seguito e garantito nella sua interezza, dal punto di origine fino alla destinazione finale. Nel contesto della sicurezza e delle reti, una connessione *end-to-end* assicura che i dati siano protetti e integri lungo l'intero percorso, senza modifiche o accessi non autorizzati. 19

Enforcement Processo attraverso il quale una regola o una politica viene applicata effettivamente in un sistema. Nel contesto della sicurezza applicativa, l'*enforcement* delle *policy* da parte di un *WAF* implica il blocco o la modifica delle richieste che violano determinate regole. 7

Exploit Sequenza di comandi o codice che sfrutta una vulnerabilità di un sistema informatico o di un'applicazione per compromettere la sicurezza, ottenere accesso non autorizzato o eseguire operazioni malevole. [24](#)

F5 Azienda statunitense che sviluppa soluzioni *hardware* e *software* per la sicurezza, la disponibilità e l'ottimizzazione delle applicazioni, tra cui i prodotti della famiglia *BIG-IP* e *WAF*. [1](#)

Firewall Sistema *hardware*, *software* o misto, progettato per monitorare e controllare il traffico di rete in entrata e in uscita in base a regole di sicurezza predefinite. Un *firewall* viene utilizzato per proteggere le reti da accessi non autorizzati e da attacchi esterni. I *WAF* rappresentano una tipologia specializzata di *firewall* applicativo. [1](#)

Framework Insieme strutturato di componenti *software*, librerie e regole che forniscono una base riutilizzabile per lo sviluppo di applicazioni. Un *framework* definisce un'architettura e offre funzionalità comuni, riducendo il tempo di sviluppo e garantendo coerenza. [6](#)

GET Metodo del protocollo *HTTP* utilizzato per richiedere una risorsa dal *server*. Le richieste *GET* dovrebbero essere idempotenti e non modificare lo stato del *server*; i parametri vengono generalmente passati nell'*URL*. [8](#)

Hardening Processo di messa in sicurezza di un sistema informatico attraverso la riduzione della sua superficie di attacco. L'*hardening* include attività come la disabilitazione di servizi non necessari, l'applicazione di *patch* di sicurezza, la configurazione di *firewall* e l'implementazione di controlli di accesso rigorosi. [24](#)

Header Insieme di informazioni aggiuntive incluse in una richiesta o risposta *HTTP*. Gli *header* specificano dettagli come il tipo di contenuto, la lunghezza, le credenziali di autenticazione o il comportamento della *cache*. Sono fondamentali per il funzionamento dei protocolli *web* e la gestione della comunicazione tra *client* e *server*. [5](#)

Hijacking Tecnica di attacco informatico in cui un attore malevolo prende il controllo di una sessione, di una connessione o di una comunicazione legittima tra due parti. Tipici esempi includono il *session hijacking*, il *browser hijacking* o il *DNS hijacking*, che mirano a intercettare o deviare il traffico per sottrarre informazioni sensibili o compromettere il sistema. [28](#)

Host Dispositivo o nodo identificabile in rete, dotato di un proprio indirizzo *IP*, che può inviare o ricevere dati. In una rete informatica, un *host* può essere un *computer*, un *server*, una macchina virtuale o qualsiasi altro sistema capace di comunicare tramite protocolli di rete. [33](#)

HTTP Protocollo di livello applicativo usato per la trasmissione di documenti ipertestuali (come le pagine *web*) su *Internet*. È il protocollo su cui si basa il *www*. [38](#)

HttpOnly Attributo di un *cookie* che impedisce l'accesso al *cookie* tramite *JavaScript* lato *client*, riducendo il rischio di furto tramite attacchi di tipo *XSS*. [8](#)

- HTTPS** Estensione sicura di *HTTP*. *HTTPS* impiega protocolli di cifratura per garantire la riservatezza e l'integrità dei dati trasmessi tra il *client* e il *server*. 38
- IP** Protocollo di comunicazione utilizzato per l'inoltro e l'instradamento dei pacchetti dati attraverso le reti informatiche. Ogni dispositivo connesso a una rete basata su *IP* è identificato da un indirizzo univoco denominato indirizzo *IP*. 38
- IP Intelligence** Tecnica di analisi che consiste nel raccogliere e utilizzare informazioni contestuali su indirizzi *IP* (es. reputazione, geolocalizzazione, attività sospette) per prendere decisioni di sicurezza. Nei *WAF*, l'*IP Intelligence* consente di bloccare richieste provenienti da *IP* noti per attività malevole o da aree geografiche a rischio. 14
- JavaScript** Linguaggio di programmazione interpretato, principalmente utilizzato per lo sviluppo di funzionalità dinamiche e interattive nelle pagine *web* lato *client*. È uno dei linguaggi fondamentali del *www* insieme a *HTML*. 9
- JSON** Formato di interscambio dati leggero, basato su testo e derivato dalla sintassi di *JavaScript*. *JSON* è ampiamente utilizzato nelle applicazioni *web* per lo scambio di dati tra *client* e *server* grazie alla sua semplicità e leggibilità. 38
- Juice Shop** Applicazione *web* vulnerabile progettata per scopi di formazione e *test* nel campo della *cybersecurity*. Consente di simulare e analizzare attacchi contro applicazioni *web*, supportando l'apprendimento pratico delle tecniche di protezione. 9
- LDAP** Protocollo applicativo aperto e indipendente dalla piattaforma, utilizzato per accedere e gestire servizi di *directory* distribuiti su rete *IP*. Il *LDAP* è comunemente impiegato per autenticazione, autorizzazione e memorizzazione di informazioni su utenti, gruppi e risorse di rete. 38
- Log** Registro strutturato contenente eventi, messaggi o attività registrate da un sistema informatico. I *log* sono fondamentali per il monitoraggio della sicurezza, la diagnosi di problemi e la verifica del comportamento delle applicazioni. 5
- Message Injection** Tecnica di attacco in cui un attore malevolo introduce messaggi o comandi non autorizzati all'interno di un flusso di comunicazione legittimo, con l'obiettivo di alterare il comportamento del sistema, ottenere accesso a dati sensibili o compromettere la sicurezza dell'applicazione. 3
- Middleware** Strato *software* che si interpone tra il sistema operativo e le applicazioni, o tra più applicazioni, facilitando la comunicazione e la gestione dei dati. Nel contesto delle applicazioni *web*, il *middleware* viene spesso utilizzato per gestire autenticazione, *logging*, validazione delle richieste o altre funzionalità trasversali. 26
- MongoDB** *Database NoSQL* orientato ai documenti, che memorizza i dati in formato *JSON* o *BSON*. Offre flessibilità nello schema e facilità di scalabilità orizzontale. 9
- Nginx** *Server web* e *reverse proxy* ad alte prestazioni, utilizzato anche come *load balancer* e *server* di *cache*. È noto per la sua efficienza nella gestione di un elevato numero di connessioni simultanee. 24

- NodeGoat** Applicazione *web* vulnerabile, progettata per scopi didattici e di ricerca nell'ambito della sicurezza applicativa. Viene utilizzata per studiare e testare vulnerabilità comuni e relative contromisure. [7](#)
- Node.js** Ambiente di runtime *JavaScript* basato sul motore V8 di *Google*, progettato per eseguire codice *JavaScript* lato *server*. *Node.js* è noto per il suo modello *event-driven* e *non-blocking I/O*, adatto allo sviluppo di applicazioni scalabili. [9](#)
- NoSQL Injection** Tipo di attacco che sfrutta vulnerabilità nelle *query* verso un database *NoSQL*, come *MongoDB*, inserendo dati malevoli per manipolare o accedere a informazioni non autorizzate. [9](#)
- Obfuscation** Tecnica utilizzata per rendere il codice sorgente o i dati difficili da leggere o interpretare, pur mantenendo la loro funzionalità originale. L'*obfuscation* è spesso impiegata per proteggere la proprietà intellettuale o ostacolare il *reverse engineering*. [20](#)
- OPTIONS** Metodo *HTTP* che consente al *client* di ottenere dal *server* le opzioni di comunicazione disponibili, come i metodi supportati su una risorsa. È spesso usato nei meccanismi di *CORS* (*Cross-Origin Resource Sharing*). [24](#)
- OSI** Modello concettuale a sette livelli sviluppato dall'*ISO* per standardizzare le funzioni di comunicazione di un sistema di rete. Il modello *OSI* suddivide le comunicazioni in livelli: fisico, collegamento dati, rete, trasporto, sessione, presentazione e applicazione. [38](#)
- OWASP** Organizzazione *no-profit* che promuove la sicurezza delle applicazioni *web* attraverso progetti *open-source*, linee guida e *standard* come l'*OWASP Top 10*, che elenca le vulnerabilità più critiche nelle applicazioni *web*. [38](#)
- Path Traversal** Vulnerabilità di sicurezza che consente a un attaccante di accedere a *file* o *directory* al di fuori della cartella prevista dall'applicazione, manipolando i percorsi nei parametri di *input*. Gli attacchi di *path traversal* possono portare alla lettura di *file* sensibili, alla divulgazione di informazioni o all'esecuzione non autorizzata di codice. [16](#)
- Payload** Parte effettiva e significativa di un pacchetto di dati o di un messaggio trasmesso in rete, che contiene le informazioni destinate all'applicazione o al destinatario finale. Nel contesto della sicurezza informatica, il termine *payload* può indicare anche il contenuto malevolo trasportato da un attacco, come uno *SQLi* o un *XSS*. [5](#)
- Pipeline** Sequenza di fasi automatizzate che consentono di sviluppare, testare, integrare e distribuire un'applicazione *software*. Nel contesto di *CI* e *CD*, una *pipeline* definisce il flusso di lavoro che va dal *commit* del codice fino al rilascio in produzione. [6](#)
- Policy** Insieme di regole configurate in un sistema (ad esempio un *WAF*) che determinano il comportamento di protezione e le azioni da intraprendere in risposta al traffico applicativo. [7](#)

- Pool** Insieme logico di *server* o *host* che collaborano per erogare un servizio condiviso. Nel contesto del *WAF* di F5, una *pool* rappresenta un gruppo di nodi che ricevono richieste dal *virtual server* e le gestiscono secondo criteri di bilanciamento del carico. [12](#)
- PortSwigger** Azienda di sicurezza informatica nota per lo sviluppo di *BurpSuite*, una *suite* di strumenti per il *penetration testing* e l'analisi della sicurezza delle applicazioni *web*. *PortSwigger* fornisce anche risorse formative come il *Web Security Academy*. [9](#)
- POST** Metodo del protocollo *HTTP* utilizzato per inviare dati al *server* al fine di creare o modificare una risorsa. I dati vengono generalmente inclusi nel corpo della richiesta e non visibili nell'*URL*. [8](#)
- Proxy Server** intermedio che riceve le richieste dai *client* e le inoltra ai *server* di destinazione, eventualmente applicando funzioni di filtraggio, *caching* o anonimizzazione del traffico. [9](#)
- PUT** Metodo del protocollo *HTTP* utilizzato per aggiornare o sostituire una risorsa sul *server* con i dati forniti nella richiesta. Le richieste *PUT* sono generalmente idempotenti. [24](#)
- PWA** Applicazione *web* che utilizza tecnologie moderne per offrire funzionalità avanzate tipiche delle applicazioni native, come notifiche *push*, accesso *offline* e installazione su dispositivi mobili. Le *PWA* uniscono i vantaggi delle *app* native e delle *app web*, garantendo portabilità e usabilità *cross-platform*. [38](#)
- Query** Richiesta formulata per ottenere informazioni da un sistema di gestione di basi di dati o da un sistema informativo. Nel contesto del *sql*, una *query* rappresenta un comando per interrogare o manipolare dati contenuti in un *database*. [20](#)
- Repository** Spazio centralizzato, locale o remoto, dove vengono memorizzati, gestiti e mantenuti dati o codice sorgente. Un *repository*, ad esempio su piattaforme come *GitHub* o *GitLab*, consente la collaborazione tra sviluppatori e il controllo di versione del *software*. [30](#)
- Runtime** Ambiente di esecuzione in cui un programma viene avviato e gestito. Il *runtime* comprende risorse *software* e *hardware* necessarie per l'esecuzione del codice, come librerie, interpreti o macchine virtuali, e può includere funzionalità di gestione della memoria, sicurezza e monitoraggio. [26](#)
- SameSite** Attributo di un *cookie* che controlla quando i *cookie* vengono inviati insieme alle richieste *cross-site*. Aiuta a mitigare attacchi di tipo *CSRF* limitando l'invio dei *cookie* solo in contesti di navigazione sicura. [24](#)
- Server** Dispositivo o programma che fornisce risorse, dati o servizi a uno o più *client* attraverso una rete. Nel modello *client-server*, il *server* elabora le richieste ricevute e restituisce le risposte appropriate.. [3](#)
- Shell** Interfaccia, testuale o grafica, che consente all'utente di interagire con il sistema operativo inviando comandi ed eseguendo programmi. Una *shell* può essere di tipo *command-line* (ad esempio *Bash* in ambienti *Linux*) o *graphical*, e viene spesso utilizzata dagli amministratori di sistema e dagli sviluppatori per configurazioni e automazioni. [6](#)

SIEM Sistema che centralizza, raccoglie e analizza i dati di *log* e gli eventi di sicurezza generati da dispositivi, applicazioni e infrastrutture *IT*. Un *SIEM* consente il monitoraggio in tempo reale, la correlazione degli eventi e l'identificazione di minacce o anomalie per supportare la risposta agli incidenti e la conformità normativa. 38

Signature Insieme di caratteristiche o *pattern* riconoscibili utilizzati per identificare specifici eventi o minacce in un sistema di sicurezza. Nel contesto di un *WAF*, le *signature* consentono di rilevare attacchi noti confrontando il traffico con un *database* di regole predefinite. 5

SPA Tipologia di applicazione *web* che carica una sola pagina *HTML* e aggiorna dinamicamente i contenuti tramite *JavaScript* e *API* senza richiedere il ricaricamento completo della pagina. Le *SPA* migliorano l'esperienza utente offrendo interazioni rapide e fluide, simili a quelle delle applicazioni *desktop*. 38

SQL Linguaggio *standard* utilizzato per l'interrogazione, la manipolazione e la definizione di dati all'interno di un *database* relazionale. 38

SQLi Tecnica di attacco che consiste nell'inserire comandi *SQL* malevoli in *input* apparentemente innocui dell'applicazione, allo scopo di manipolare le *query* verso il *database* sottostante, accedendo, alterando o eliminando dati sensibili. 38

SSRF Vulnerabilità in cui un attaccante induce un *server* vulnerabile a inviare richieste *HTTP* verso domini o indirizzi scelti dall'attaccante stesso. Un *SSRF* può essere sfruttato per accedere a risorse interne non esposte pubblicamente, esfiltrare dati sensibili o sfruttare altri servizi interni alla rete. 39

Staging Fase intermedia di *test* in cui le modifiche o le configurazioni vengono applicate in un ambiente controllato e non ancora attivo in produzione. Nel contesto delle *policy* di sicurezza, lo *staging* consente di valutare l'impatto delle regole senza bloccare il traffico reale. 7

Stateless Caratteristica di un sistema o protocollo in cui ogni richiesta viene trattata in modo indipendente, senza che il *server* mantenga informazioni sullo stato delle interazioni precedenti. Nei sistemi *web*, un'architettura *stateless* semplifica la scalabilità e riduce la complessità della gestione delle sessioni. 3

Supply Chain Insieme di processi e attori coinvolti nella produzione e distribuzione di un prodotto o servizio. Nel contesto informatico, la *supply chain* fa riferimento all'insieme di dipendenze *software* e componenti di terze parti che, se compromessi, possono introdurre vulnerabilità o *backdoor* all'interno dei sistemi finali. 29

Tenant Nel contesto dei sistemi *multi-tenant*, un *tenant* è un'entità organizzativa o un cliente che condivide le stesse risorse fisiche o logiche di un'infrastruttura, ma dispone di dati e configurazioni isolati rispetto agli altri. 22

Threat Intelligence Processo di raccolta, analisi e condivisione di informazioni relative a potenziali minacce informatiche. La *threat intelligence* supporta le organizzazioni nel prevenire, rilevare e rispondere ad attacchi informatici, fornendo dati su *threat actors*, tecniche di attacco, indicatori di compromissione e trend emergenti. 36

Threat Modelling Processo strutturato per identificare, analizzare e classificare le potenziali minacce alla sicurezza di un sistema, valutandone l'impatto e la probabilità. Il *threat modelling* aiuta a definire contromisure adeguate e priorità di mitigazione. [22](#)

TLS Protocollo crittografico che garantisce la sicurezza delle comunicazioni su reti informatiche. *TLS* assicura confidenzialità, integrità e autenticità dei dati trasmessi tra *client* e *server*, ed è ampiamente utilizzato in combinazione con *HTTPS* per proteggere il traffico *web*. [39](#)

Token Elemento di dati, spesso generato in modo univoco, utilizzato per rappresentare l'identità o l'autenticazione di un utente, oppure per autorizzare l'accesso a risorse specifiche. Nei sistemi *web*, i *token* vengono comunemente usati nei meccanismi di autenticazione e gestione delle sessioni. [9](#)

TRACE Metodo *HTTP* utilizzato principalmente per scopi diagnostici. Permette di ricevere dal *server* la stessa richiesta inviata dal *client*, utile per verificare eventuali modifiche o aggiunte fatte da *proxy* o altri componenti lungo il percorso. [24](#)

Tuning Processo iterativo di ottimizzazione delle *policy* di sicurezza, che consiste nell'analizzare i risultati dei *test* e dei *log* per regolare e affinare progressivamente le regole di protezione, riducendo i falsi positivi e migliorando l'efficacia del *WAF*. [12](#)

Ubuntu Distribuzione del sistema operativo *Linux* basata su *Debian*, nota per la sua facilità d'uso, il supporto a lungo termine e una vasta comunità di utenti. *Ubuntu* è disponibile in diverse versioni, tra cui *Ubuntu Server* e *Ubuntu Desktop*, ed è ampiamente utilizzata in ambienti di sviluppo, *test* e produzione. [10](#)

URI Stringa che identifica in modo univoco una risorsa su una rete. Un *URI* può essere ulteriormente classificato come *URL* (*Uniform Resource Locator*), che specifica anche il modo di accedere alla risorsa, o *URN* (*Uniform Resource Name*), che la identifica tramite un nome univoco indipendente dalla sua posizione. [39](#)

URL Indirizzo univoco che identifica la posizione di una risorsa su *Internet* e il protocollo utilizzato per accedervi. Un *URL* è composto da diverse parti, come il protocollo (*HTTP* o *HTTPS*), il nome di dominio, la porta, il percorso e, facoltativamente, parametri e frammenti. [39](#)

Virtual Server Configurazione logica in un sistema di bilanciamento del carico che permette di esporre un indirizzo *IP* e una porta a cui indirizzare il traffico in ingresso, inoltrandolo poi verso uno o più *server* reali secondo regole predefinite. [13](#)

VMware Workstation *Software* di virtualizzazione che consente di creare e gestire macchine virtuali su un *computer host*. È utilizzato per eseguire più sistemi operativi isolati simultaneamente in un ambiente virtuale. [10](#)

WAF Sistema di protezione che monitora, filtra e analizza il traffico *HTTP/HTTPS* verso e da una applicazione *web*, con l'obiettivo di proteggere da attacchi noti e sconosciuti come *SQLi*, *XSS* e altri attacchi a livello applicativo. [39](#)

Web Scraping Tecnica automatizzata per estrarre grandi quantità di dati da siti *web*, spesso tramite *bot* o *script*. Sebbene possa avere utilizzi legittimi, il *web scraping* può essere anche impiegato per attività malevole, come il furto di contenuti o la raccolta non autorizzata di dati sensibili. [5](#)

WebSocket Protocollo di comunicazione che fornisce un canale bidirezionale e *full-duplex* tra un *client* e un *server* attraverso una singola connessione *TCP*. I *WebSocket* consentono interazioni in tempo reale con bassa latenza, spesso utilizzati in applicazioni come *chat*, giochi *online* e sistemi di monitoraggio. [3](#)

X-Frame-Options *Header* di sicurezza *HTTP* che specifica se una pagina può essere caricata all'interno di un *frame*, *iframe* o *object*. È utilizzato per prevenire attacchi di tipo *clickjacking*. [24](#)

XML Linguaggio di *markup* flessibile e strutturato utilizzato per rappresentare e scambiare dati in formato leggibile sia da umani che da macchine. *XML* consente di definire documenti personalizzati tramite *tag* e viene utilizzato in numerosi *standard* e protocolli di comunicazione. [39](#)

XPath Linguaggio di *query* utilizzato per navigare tra elementi e attributi in documenti *XML*. *XPath* è spesso impiegato in combinazione con *XSLT* o *XQuery* per filtrare, selezionare e manipolare dati strutturati. [39](#)

XSS Tipologia di vulnerabilità delle applicazioni *web* che consente a un attaccante di iniettare codice *JavaScript* o *HTML* malevolo nelle pagine visualizzate da altri utenti, con lo scopo di rubare dati sensibili, sessioni utente o manipolare il contenuto della pagina. [39](#)

XXE Vulnerabilità di sicurezza che sfrutta il trattamento improprio di entità esterne nei documenti *XML*. Un attaccante può abusare di questa tecnica per accedere a *file* locali, eseguire attacchi *DoS* o ottenere informazioni sensibili. [39](#)